



北京大學

第二讲 神经语言模型与词嵌入

基于深度学习的自然语言理解

邓志鸿

2025年9月17日星期三



- 神经语言模型

- ✓背景

- ✓前馈神经语言模型

- ✓模型训练

- 词嵌入

- ✓背景

- ✓向量语义

- ✓Word2vec

- 讨论

• N-gram语言模型的主要问题

✓稀疏问题

Sparsity Problem 1

Problem: What if “students opened their w ” never occurred in data? Then w has probability 0!

(Partial) Solution: Add small δ to the count for every $w \in V$. This is called *smoothing*.

$$P(w|\text{students opened their}) = \frac{\text{count}(\text{students opened their } w)}{\text{count}(\text{students opened their})}$$

Sparsity Problem 2

Problem: What if “students opened their” never occurred in data? Then we can’t calculate probability for *any* w !

(Partial) Solution: Just condition on “opened their” instead. This is called *backoff*.

Note: Increasing n makes sparsity problems *worse*. Typically, we can’t have n bigger than 5.

- N-gram语言模型的主要问题

- ✓ 存储问题

Storage: Need to store count for all n -grams you saw in the corpus.

$$P(\mathbf{w} | \text{students opened their}) = \frac{\text{count}(\text{students opened their } \mathbf{w})}{\text{count}(\text{students opened their})}$$

Increasing n or increasing corpus increases model size!

- 统计语言模型

$$P(t_l | t_{l-n}, t_{l-n+1}, \dots, t_{l-1}) = \frac{P(t_{l-n}, t_{l-n+1}, \dots, t_l)}{P(t_{l-n}, t_{l-n+1}, \dots, t_{l-1})} = \frac{\text{Count}(t_{l-n}, t_{l-n+1}, \dots, t_l)}{\text{Count}(t_{l-n}, t_{l-n+1}, \dots, t_{l-1})}$$

- 神经网络语言模型 (Neural Language Modelling)

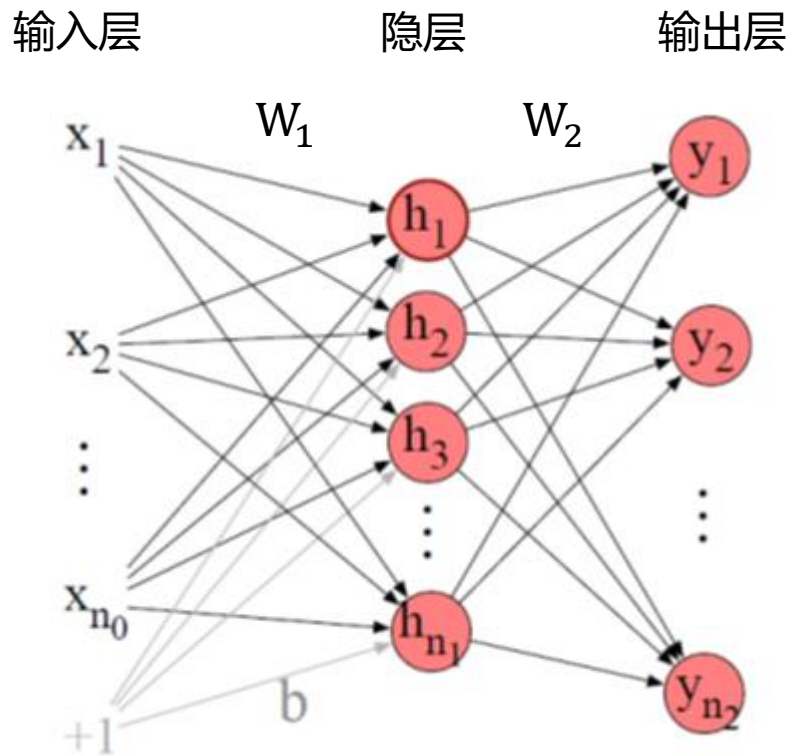
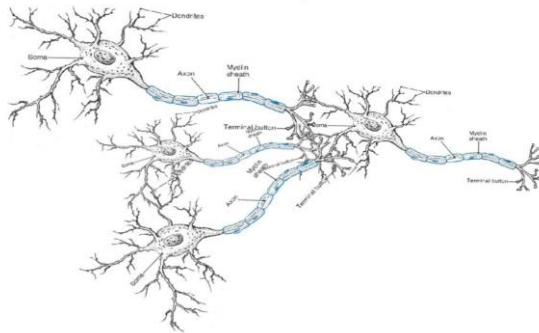
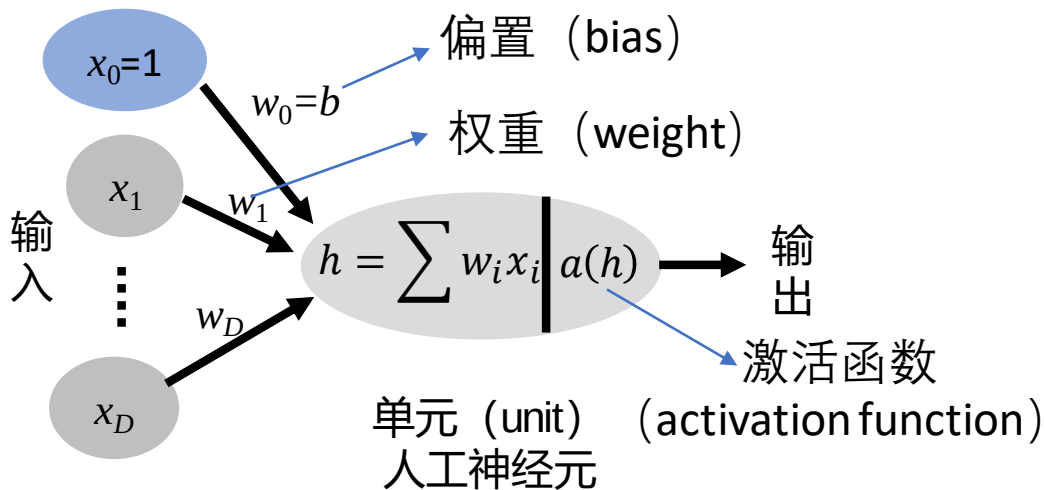
$$P(t_l | t_{l-n}, t_{l-n+1}, \dots, t_{l-1}) = f(t_{l-n}, t_{l-n+1}, \dots, t_{l-1})$$

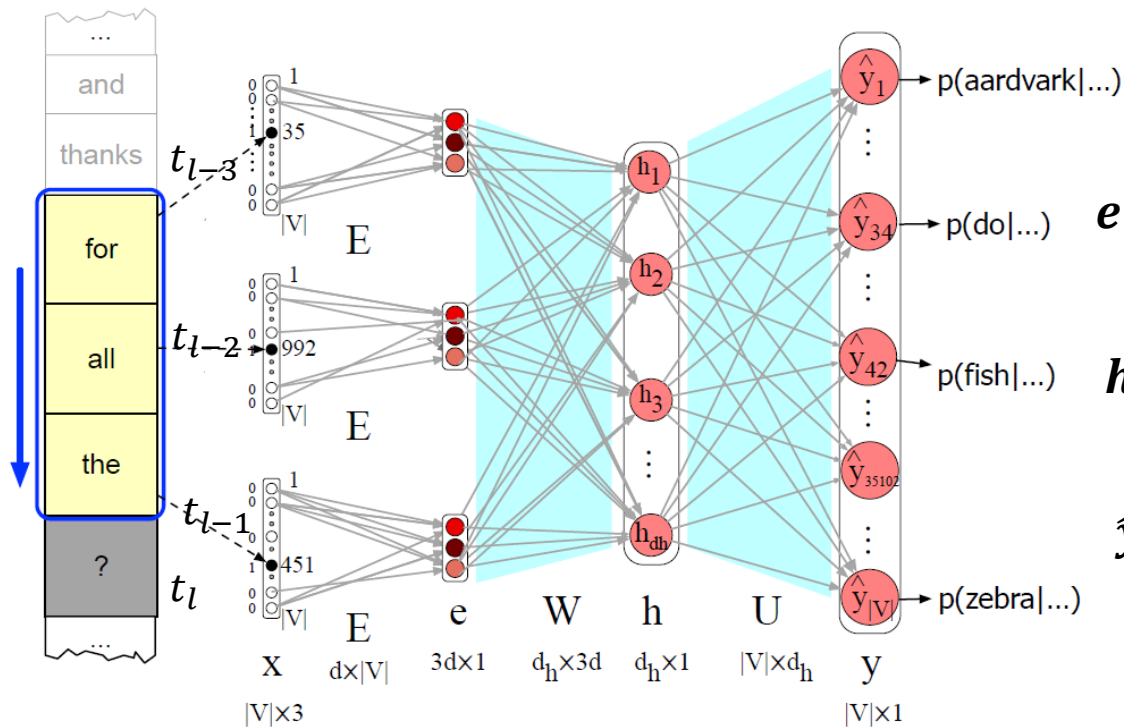


神经网络

- Neural Language Model
 - ✓采用神经网络来计算语言模型的概率
 - ✓相比N-gram语言模型神经语言模型能处理更长的序列，更好的泛化能力，更好的性能，以及更加灵活。当然，神经语言模型更为复杂、训练困难、可解释性差
- 代表性模型
 - ✓前馈神经语言模型（Feedforward Neural Language Model）
 - ✓循环神经网络语言模型（Recurrent Neural Networks）
 - ✓基于Transformer的语言模型

- 前馈神经网络 (feedforward neural network)
 - ✓ 也称多层感知器 (multilayer perceptron, MLP)
 - ✓ 除输入层和输出层外，还有一到多个隐层，输入层通常被称为第0层
 - ✓ 每个隐层单元采用可微分函数作为激活函数
 - ✓ 采用基于反向传播的梯度下降算法学习网络权重
- 前馈神经网络语言模型
 - ✓ 基于前馈神经网络，由Bengio等人于2003年提出
 - 本讲介绍Bengio等人提出模型的简化版本
 - ✓ 模型简单，但其提出的概念（如词向量表示）对后续的神经语言模型产生深远的影响





input layer
one-hot
vectors

输入层

embedding
layer

嵌入层

hidden
layer

隐层

output layer
softmax

输出层

$$\mathbf{e} = [Et_{l-1}; Et_{l-2}; Et_{l-3}]$$

可以是其他激活函数

$$\mathbf{h} = \tanh(\mathbf{W}\mathbf{e} + \mathbf{b}) \quad \mathbf{b} \in \mathbb{R}^{d_h \times 1}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{c}) \quad \mathbf{c} \in \mathbb{R}^{|V| \times 1}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \quad \tanh(\mathbf{x}) = \begin{bmatrix} \tanh(x_1) \\ \vdots \\ \tanh(x_n) \end{bmatrix}$$

$$\tanh(x_i) = \frac{\exp(x_i) - \exp(-x_i)}{\exp(x_i) + \exp(-x_i)}$$

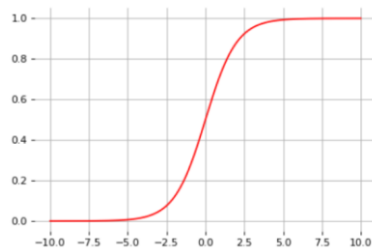
$$\frac{\partial \tanh(x_i)}{\partial x_i} = 1 - \tanh(x_i)^2 \quad ?$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} \quad \text{softmax}(\mathbf{x}) = \begin{bmatrix} \text{softmax}(x_1) \\ \vdots \\ \text{softmax}(x_n) \end{bmatrix}$$

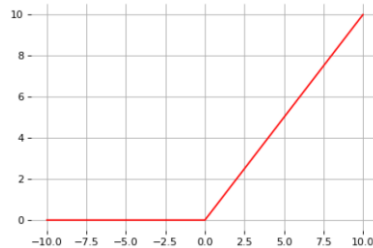
$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

$$\frac{\partial \text{softmax}(x_i)}{\partial x_i} = \text{softmax}(x_i)(1 - \text{softmax}(x_i)) \quad ?$$

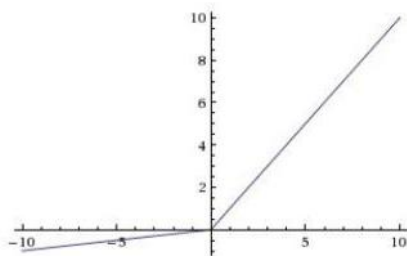
$$\sigma(u) = \frac{1}{1 + \exp(-u)}$$



$$\text{ReLU}(u) = \max(0, u)$$



$$\text{LReLU}(u; \alpha) = \max(\alpha u, u) = \begin{cases} \alpha u, & \text{if } u \leq 0 \\ u, & \text{if } u > 0 \end{cases}$$



$$\text{ELU}(u; \alpha) = \begin{cases} \alpha(e^{-u} - 1), & \text{if } u \leq 0 \\ u, & \text{if } u > 0 \end{cases}$$

$$\text{SELU}(u; \lambda) = \lambda \text{ELU}(u)$$

$$\text{GELU}(u) = u\Phi(u)$$

$$\Phi(u) = P_{x \sim \mathcal{N}(0,1)}(x \leq u)$$

$$\theta = \{E, W, U, \mathbf{b}, \mathbf{c}\}$$

$$\mathbf{e} = [Et_{l-1}; Et_{l-2}; Et_{l-3}] \quad E \in \mathbb{R}^{d \times |V|}$$

$$\mathbf{h} = \tanh(W\mathbf{e} + \mathbf{b}) \quad W \in \mathbb{R}^{d_h \times 3d} \quad \mathbf{b} \in \mathbb{R}^{d_h \times 1}$$

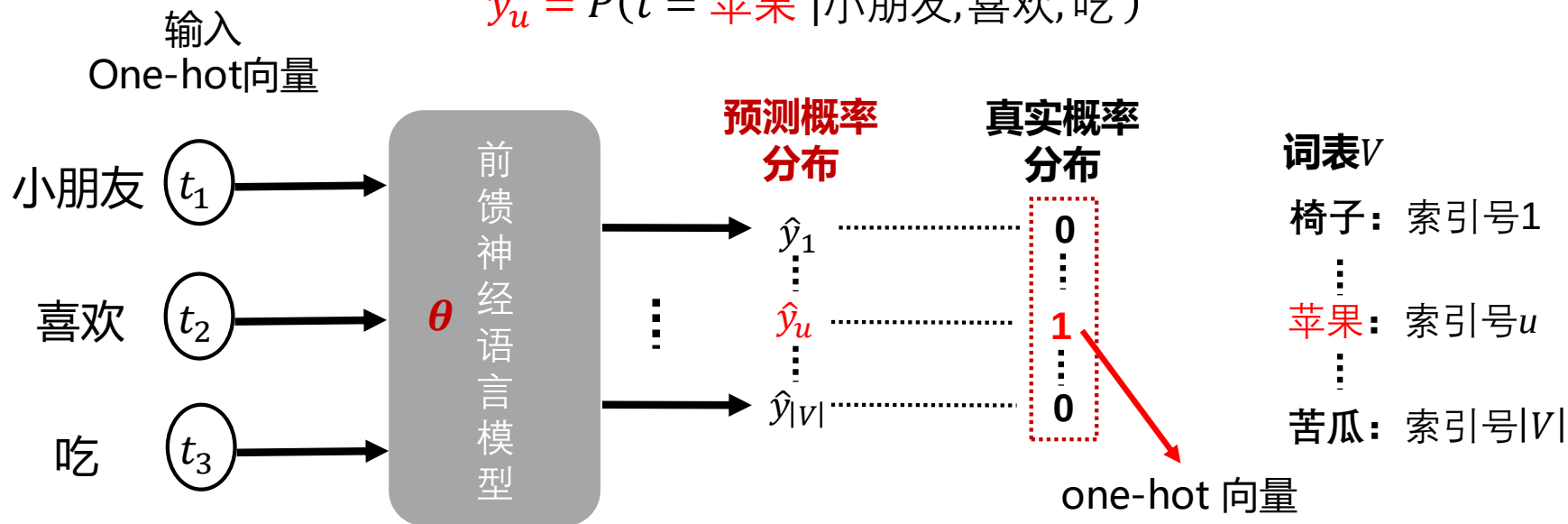
$$\hat{\mathbf{y}} = \text{softmax}(U\mathbf{h} + \mathbf{c}) \quad U \in \mathbb{R}^{|V| \times d_h} \quad \mathbf{c} \in \mathbb{R}^{|V| \times 1}$$

模型参数量?

训练文本：小朋友喜欢吃苹果

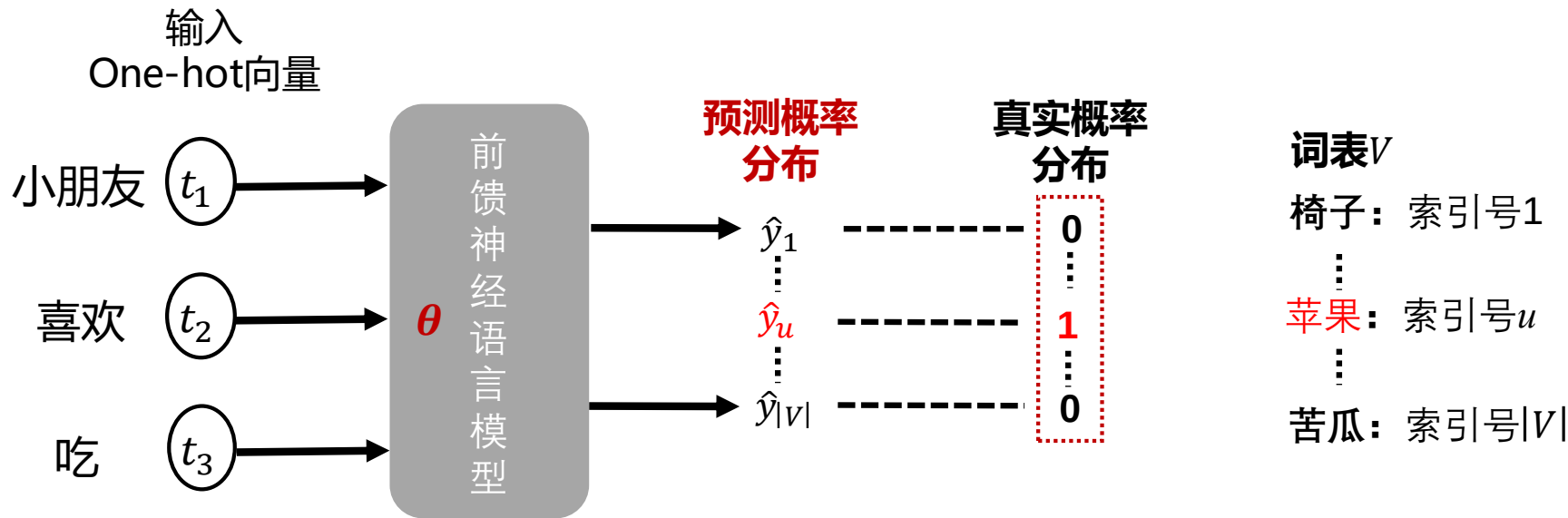
目标：预测概率分布 $P(t_4 | \text{小朋友, 喜欢, 吃})$ 与真实概率分布尽可能一致

$$\hat{y}_u = P(t = \text{苹果} | \text{小朋友, 喜欢, 吃})$$



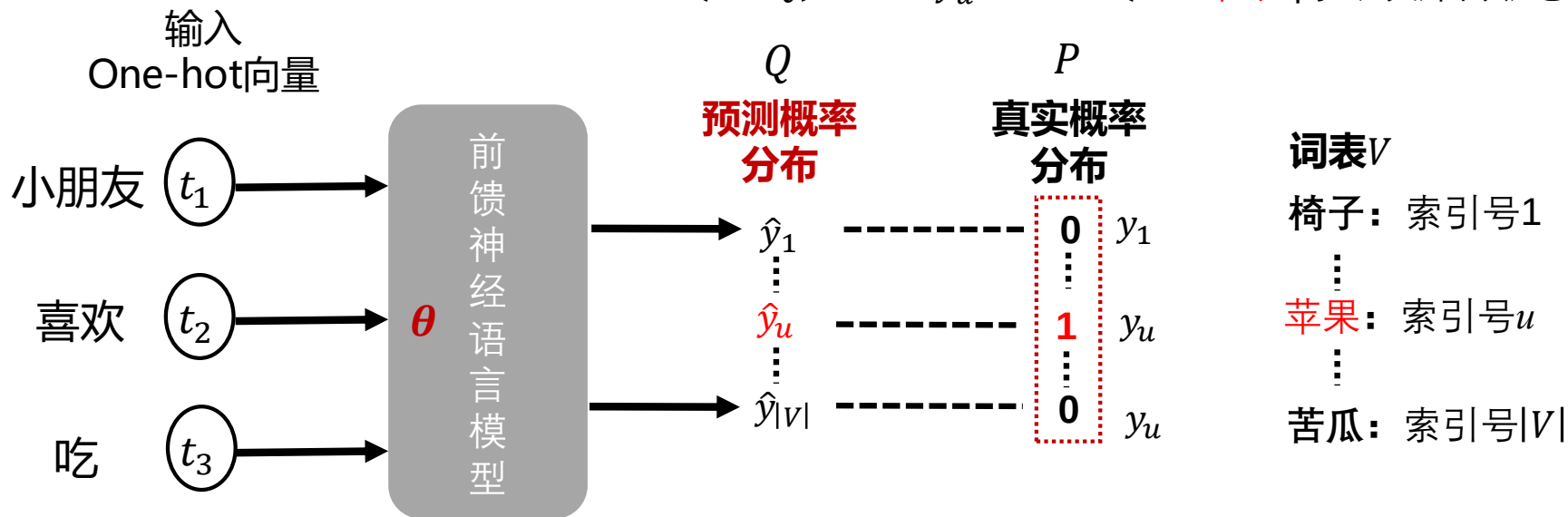
度量概率分布的经典方法：KL散度

$$\text{KL}(P \parallel Q) \triangleq \sum_i p_i \log \frac{p_i}{q_i} = - \sum_i p_i \log \frac{q_i}{p_i} = \mathbb{E}_{p_i \sim P} \left[\log \frac{p_i}{q_i} \right]$$



$$\text{KL}(P \parallel Q) = - \sum_{i=1}^{|V|} y_i \log \frac{\hat{y}_i}{y_i} = -\log \hat{y}_u$$

$$\text{MinKL}(P \parallel Q) = \text{Max} \hat{y}_u = \text{Max} P(t = \text{苹果} \mid \text{小朋友, 喜欢, 吃})$$



$$\text{KL}(P \parallel Q) = - \sum_{i=1}^{|V|} y_i \log \frac{\hat{y}_i}{y_i} = -\log \hat{y}_u$$

$$\underset{\theta}{\text{Min}} \text{KL}(P \parallel Q) = \underset{\theta}{\text{Max}} \hat{y}_u = \underset{\theta}{\text{Max}} P(t = \text{苹果} | \text{小朋友, 喜欢, 吃})$$

给定训练语料集合 $\mathcal{D} = \{T_n\}_{n=1}^N$, $T_n = t_{n1}t_{n2} \dots t_{nk}t_{n(k+1)}$, 模型训练损失函数为 $\sum_{n=1}^N \text{KL}(P_n \parallel Q_n)$

$$\underset{\theta}{\text{Min}} \sum_{n=1}^N \text{KL}(P_n \parallel Q_n) = \underset{\theta}{\text{Min}} \sum_{n=1}^N -\log \hat{y}_{n_u}$$

负对数似然
NLL, Negative Log-Likelihood

$$\underset{\theta}{\text{Max}} \prod_{n=1}^N P(t = t_{n(k+1)} | t_{n1}t_{n2} \dots t_{nk})$$

$$\hat{y}_{n_u} = P(t = t_{n(k+1)} | t_{n1}t_{n2} \dots t_{nk})$$

Input: $\mathcal{D} = \{T_n\}_{n=1}^N$; ρ , the learning rate

Output: weights θ

学习率，可以是固定的，也可以是变化的，深度学习常用Adam等自适应学习率

Initialize θ ; //to a vector with some small random value in each dimension

Until the terminating condition is met, do

$$\Delta\theta \leftarrow 0$$

For each $T_n \in \mathcal{D}$, do

$$\Delta\theta \leftarrow \Delta\theta + \nabla_{\theta}(-\log \hat{y}_{n_u})$$

} ?

$$\theta \leftarrow \theta - \rho \frac{1}{N} \Delta\theta$$

?

Input: $\mathcal{D} = \{T_n\}_{n=1}^N$; ρ , the learning rate 梯度下降要处理完所有训练数据才更新一次权重，内存占用大，学习效率低。在神经网络/深度学习领域，通常用Minibatch梯度下降算法进行模型训练

Output: weights θ

Initialize θ ; //to a vector with some small random value in each dimension

Until **the terminating condition** is met, do

$$\Delta\theta \leftarrow 0$$

Construct $\mathcal{D}^*$ by sampling B ($\ll N$) training examples from $\mathcal{D}$

For each $T_n \in \mathcal{D}^*$, do

$$\Delta\theta \leftarrow \Delta\theta + \nabla_{\theta}(-\log\hat{y}_{n_u})$$

$$\theta \leftarrow \theta - \rho \frac{1}{B} \Delta\theta$$

$$\mathbf{g}_t = \frac{1}{K} \nabla_{\boldsymbol{\theta}} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_K} J(\boldsymbol{\theta}_{t-1}, \mathbf{x}, \mathbf{y})$$

$$\mathbf{a}_t = \mathbf{a}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t \quad \mathbf{a}_0 = \mathbf{0} \text{ (零向量)}$$

$$\Delta \boldsymbol{\theta}_t = - \frac{\eta}{\sqrt{\mathbf{a}_t + \delta \mathbf{I}}} \odot \mathbf{g}_t$$

自适应学习率

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} + \Delta \boldsymbol{\theta}_t$$

$$\mathbf{x} \triangleq \begin{bmatrix} x_1 \\ \vdots \\ x_D \end{bmatrix} \quad \mathbf{y} \triangleq \begin{bmatrix} y_1 \\ \vdots \\ y_D \end{bmatrix}$$

$$\mathbf{x} \odot \mathbf{y} = \begin{bmatrix} x_1 \times y_1 \\ \vdots \\ x_D \times y_D \end{bmatrix}$$

$$\mathbf{a}_t \triangleq \begin{bmatrix} a_{t1} \\ \vdots \\ a_{tC} \end{bmatrix} \quad \frac{\eta}{\sqrt{\mathbf{a}_t + \delta \mathbf{I}}} = \begin{bmatrix} \frac{\eta}{\sqrt{a_{t1} + \delta}} \\ \vdots \\ \frac{\eta}{\sqrt{a_{tC} + \delta}} \end{bmatrix}$$

$$\mathbf{g}_t = \frac{1}{K} \nabla_{\boldsymbol{\theta}} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_K} J(\boldsymbol{\theta}_{t-1}, \mathbf{x}, \mathbf{y})$$

$$\mathbf{a}_t = \lambda \mathbf{a}_{t-1} + (1 - \lambda) \mathbf{g}_t \odot \mathbf{g}_t$$

$$\Delta \boldsymbol{\theta}_t = - \frac{\eta}{\sqrt{\mathbf{a}_t + \delta \mathbf{I}}} \odot \mathbf{g}_t$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} + \Delta \boldsymbol{\theta}_t$$

自适应学习率

$$\mathbf{g}_t = \frac{1}{K} \nabla_{\boldsymbol{\theta}} \sum_{(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_K} J(\boldsymbol{\theta}_{t-1}, \mathbf{x}, \mathbf{y})$$

$$\mathbf{s}_t = \lambda_1 \mathbf{s}_{t-1} + (1 - \lambda_1) \mathbf{g}_t$$

$$\mathbf{r}_t = \lambda_2 \mathbf{r}_{t-1} + (1 - \lambda_2) \mathbf{g}_t \odot \mathbf{g}_t$$

$$\tilde{\mathbf{s}}_t = \frac{\mathbf{s}_t}{1 - \lambda_1^t} \quad \tilde{\mathbf{r}}_t = \frac{\mathbf{r}_t}{1 - \lambda_2^t}$$

$$\Delta \boldsymbol{\theta}_t = - \frac{\eta}{\sqrt{\tilde{\mathbf{r}}_t} + \delta \mathbf{I}} \odot \tilde{\mathbf{s}}_t$$

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} + \Delta \boldsymbol{\theta}_t$$

自适应学习率

$$\mathbf{s}_0 = \mathbf{r}_0 = \mathbf{0}$$

$$\tilde{\mathbf{r}}_t \triangleq \begin{bmatrix} \tilde{r}_{t1} \\ \vdots \\ \tilde{r}_{tC} \end{bmatrix}$$

$T_n = t_{n1}t_{n2} \dots t_{nk}t_{n(k+1)}$, 输入 $t_{n1}t_{n2} \dots t_{nk}$, 输出在词表上的概率分布 $P(t|t_{n1}t_{n2} \dots t_{nk})$

损失函数: $-\log \hat{y}_u$

$$\theta = \{E, W, U, \mathbf{b}, \mathbf{c}\}$$

$$\mathbf{e} = [Et_{n1}; Et_{n2}; \dots; t_{nk}] \quad E \in \mathbb{R}^{d \times |V|}$$

$$\mathbf{h} = \tanh(W\mathbf{e} + \mathbf{b}) \quad W \in \mathbb{R}^{d_h \times kd} \quad \mathbf{b} \in \mathbb{R}^{d_h \times 1}$$

$$\hat{\mathbf{y}} = \text{softmax}(U\mathbf{h} + \mathbf{c}) \quad U \in \mathbb{R}^{|V| \times d_h} \quad \mathbf{c} \in \mathbb{R}^{|V| \times 1}$$

$$\nabla_{\theta}(-\log \hat{y}_{n_u})$$



$$\nabla_E(-\log \hat{y}_{n_u})$$

$$\nabla_W(-\log \hat{y}_{n_u})$$

$$\nabla_{\mathbf{b}}(-\log \hat{y}_{n_u})$$

$$\nabla_U(-\log \hat{y}_{n_u})$$

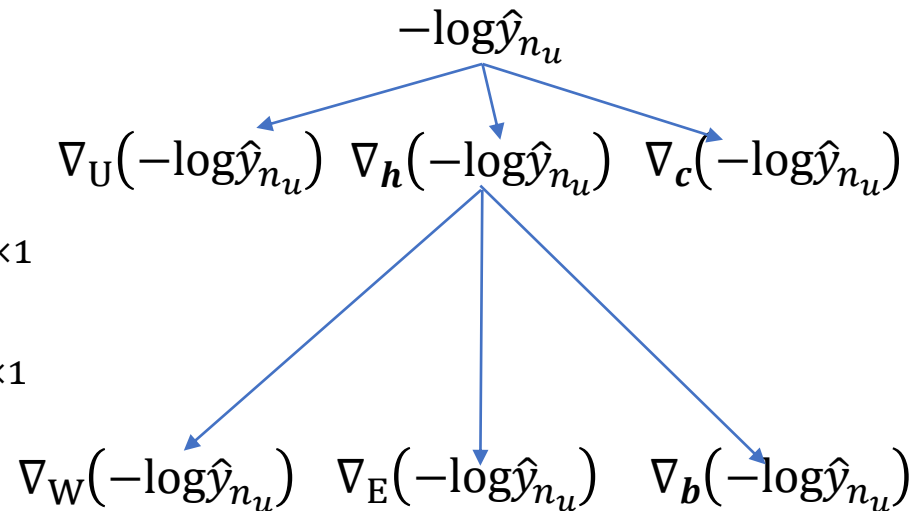
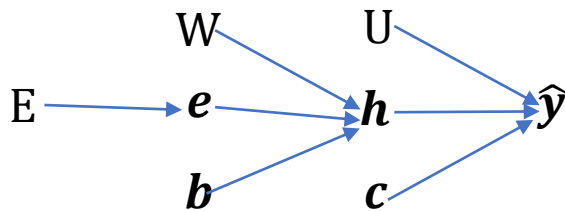
$$\nabla_{\mathbf{c}}(-\log \hat{y}_{n_u})$$

$$\theta = \{E, W, U, \mathbf{b}, \mathbf{c}\}$$

$$\mathbf{e} = [Et_{n1}; Et_{n2}; \dots; t_{nk}] \quad E \in \mathbb{R}^{d \times |V|}$$

$$\mathbf{h} = \tanh(W\mathbf{e} + \mathbf{b}) \quad W \in \mathbb{R}^{d_h \times kd} \quad \mathbf{b} \in \mathbb{R}^{d_h \times 1}$$

$$\hat{\mathbf{y}} = \text{softmax}(U\mathbf{h} + \mathbf{c}) \quad U \in \mathbb{R}^{|V| \times d_h} \quad \mathbf{c} \in \mathbb{R}^{|V| \times 1}$$



链式法则：要计算 $\nabla_W(-\log \hat{y}_{n_u})$ ，必须先计算 $\nabla_h(-\log \hat{y}_{n_u})$
 通过链式法则求梯度（偏导）的方法称为反向传播方法

$$\mathbf{c} = \begin{bmatrix} c_1 \\ \vdots \\ c_{|V|} \end{bmatrix} \in \mathbb{R}^{|V| \times 1} \quad \mathbf{h} = \begin{bmatrix} h_1 \\ \vdots \\ h_{d_h} \end{bmatrix} \in \mathbb{R}^{d_h \times 1} \quad \mathbf{U} = \begin{bmatrix} u_{11} & \cdots & u_{1d_h} \\ \vdots & \ddots & \vdots \\ u_{|V|1} & \cdots & u_{|V|d_h} \end{bmatrix} \in \mathbb{R}^{|V| \times d_h} \quad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_{|V|} \end{bmatrix} \in \mathbb{R}^{|V| \times 1}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{c})$$

$$o_i = \sum_{j=1}^{d_h} u_{ij} h_j + c_i$$

$$\hat{y}_i = \text{softmax}(o_i) = \frac{\exp(o_i)}{\sum_j \exp(o_j)}$$

$$\nabla_{\mathbf{c}}(-\log \hat{y}_{n_u}) = \begin{bmatrix} \frac{\partial -\log \hat{y}_{n_u}}{\partial c_1} \\ \vdots \\ \frac{\partial -\log \hat{y}_{n_u}}{\partial c_{|V|}} \end{bmatrix}$$

$$\begin{aligned} \frac{\partial -\log \hat{y}_{n_u}}{\partial c_i} &= \frac{\partial -\log \hat{y}_{n_u}}{\partial \hat{y}_{n_u}} \times \frac{\partial \hat{y}_{n_u}}{\partial o_i} \times \frac{\partial o_i}{\partial c_i} \\ &= -\frac{1}{\hat{y}_{n_u}} \times \frac{\partial \hat{y}_{n_u}}{\partial o_i} \end{aligned}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{c})$$

$$o_i = \sum_{j=1}^{d_h} u_{ij} h_j + c_i \qquad \hat{y}_{n_u} = \text{softmax}(o_{n_u}) = \frac{\exp(o_{n_u})}{\sum_j \exp(o_j)}$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial c_i} = -\frac{1}{\hat{y}_{n_u}} \times \frac{\partial \hat{y}_{n_u}}{\partial o_i} = \begin{cases} \hat{y}_{n_u} - 1 & , n_u = i \\ \hat{y}_i & , n_u \neq i \end{cases} = \hat{y}_i - \mathbf{I}(n_u = i)$$



$$\frac{\partial \hat{y}_{n_u}}{\partial o_i} = \begin{cases} \hat{y}_{n_u}(1 - \hat{y}_{n_u}) & , n_u = i \\ -\hat{y}_{n_u}\hat{y}_i & , n_u \neq i \end{cases} = \hat{y}_{n_u}(\mathbf{I}(n_u = i) - \hat{y}_i) \quad ?$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{c})$$

$$o_i = \sum_{j=1}^{d_h} u_{ij} h_j + c_i \quad \hat{y}_{n_u} = \text{softmax}(o_{n_u}) = \frac{\exp(o_{n_u})}{\sum_j \exp(o_j)}$$

$$\begin{aligned} \nabla_{\mathbf{h}}(-\log \hat{y}_{n_u}) &= \begin{bmatrix} \frac{\partial -\log \hat{y}_{n_u}}{\partial h_1} \\ \vdots \\ \frac{\partial -\log \hat{y}_{n_u}}{\partial h_{d_h}} \end{bmatrix} \\ \frac{\partial -\log \hat{y}_{n_u}}{\partial h_j} &= \frac{\partial -\log \hat{y}_{n_u}}{\partial \hat{y}_{n_u}} \times \left(\sum_{i=1}^{|V|} \frac{\partial \hat{y}_{n_u}}{\partial o_i} \times \frac{\partial o_i}{\partial h_j} \right) \\ &= -\frac{1}{\hat{y}_{n_u}} \times \left(\sum_{i=1}^{|V|} \hat{y}_{n_u} (\mathbf{I}(n_u = i) - \hat{y}_i) \times u_{ij} \right) \\ &= \sum_{i=1}^{|V|} (\hat{y}_i - \mathbf{I}(n_u = i)) \times u_{ij} \end{aligned}$$

$$\mathbf{c} = \begin{bmatrix} c_1 \\ \vdots \\ c_{|V|} \end{bmatrix} \in \mathbb{R}^{|V| \times 1} \quad \mathbf{h} = \begin{bmatrix} h_1 \\ \vdots \\ h_{d_h} \end{bmatrix} \in \mathbb{R}^{d_h \times 1} \quad \mathbf{U} = \begin{bmatrix} u_{11} & \cdots & u_{1d_h} \\ \vdots & \ddots & \vdots \\ u_{|V|1} & \cdots & u_{|V|d_h} \end{bmatrix} \in \mathbb{R}^{|V| \times d_h} \quad \hat{\mathbf{y}} = \begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_{|V|} \end{bmatrix} \in \mathbb{R}^{|V| \times 1}$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{c})$$

$$o_i = \sum_{j=1}^{d_h} u_{ij} h_j + c_i \quad \hat{y}_{n_u} = \text{softmax}(o_{n_u}) = \frac{\exp(o_{n_u})}{\sum_j \exp(o_j)}$$

$$\nabla_{\mathbf{U}}(-\log \hat{y}_{n_u}) = \begin{bmatrix} \frac{\partial -\log \hat{y}_{n_u}}{\partial u_{11}} & \cdots & \frac{\partial -\log \hat{y}_{n_u}}{\partial u_{1d_h}} \\ \vdots & \ddots & \vdots \\ \frac{\partial -\log \hat{y}_{n_u}}{\partial u_{|V|1}} & \cdots & \frac{\partial -\log \hat{y}_{n_u}}{\partial u_{|V|d_h}} \end{bmatrix} \quad \frac{\partial -\log \hat{y}_{n_u}}{\partial u_{lm}} ?$$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{U}\mathbf{h} + \mathbf{c})$$

$$o_{\mathbf{i}} = \sum_{j=1}^{d_h} u_{\mathbf{i}j} h_j + c_i \quad \hat{y}_{n_u} = \text{softmax}(o_{n_u}) = \frac{\exp(o_{n_u})}{\sum_j \exp(o_j)}$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial u_{lm}} = \frac{\partial -\log \hat{y}_{n_u}}{\partial \hat{y}_{n_u}} \times \left(\frac{\partial \hat{y}_{n_u}}{\partial o_l} \times \frac{\partial o_l}{\partial u_{lm}} \right)$$

$$= -\frac{1}{\hat{y}_{n_u}} \times \hat{y}_{n_u} (\mathbb{I}(n_u = l) - \hat{y}_l) \times h_m$$

$$\mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_{d_h} \end{bmatrix} \in \mathbb{R}^{d_h \times 1}$$

$$\mathbf{h} = \begin{bmatrix} h_1 \\ \vdots \\ h_{d_h} \end{bmatrix} \in \mathbb{R}^{d_h \times 1}$$

$$\mathbf{e} = \begin{bmatrix} e_1 \\ \vdots \\ e_{kd} \end{bmatrix} \in \mathbb{R}^{kd \times 1}$$

$$\mathbf{W} = \begin{bmatrix} w_{11} & \cdots & w_{1kd} \\ \vdots & \ddots & \vdots \\ w_{d_h 1} & \cdots & w_{d_h kd} \end{bmatrix} \in \mathbb{R}^{d_h \times kd}$$

$$\mathbf{h} = \tanh(\mathbf{W}\mathbf{e} + \mathbf{b})$$

$$o_i = \sum_{j=1}^{kd} w_{ij} e_j + b_i$$

$$\nabla_{\mathbf{b}}(-\log \hat{y}_{n_u}) = \begin{bmatrix} \frac{\partial -\log \hat{y}_{n_u}}{\partial b_1} \\ \vdots \\ \frac{\partial -\log \hat{y}_{n_u}}{\partial b_{d_h}} \end{bmatrix}$$

$$h_i = \tanh(o_i) = \frac{\exp(o_i) - \exp(-o_i)}{\exp(o_i) + \exp(-o_i)}$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial b_i} = \frac{\partial -\log \hat{y}_{n_u}}{\partial h_i} \times \frac{\partial h_i}{\partial o_i} \times \frac{\partial o_i}{\partial b_i}$$

$$h_i = \tanh(o_i) = \frac{\exp(o_i) - \exp(-o_i)}{\exp(o_i) + \exp(-o_i)}$$

$$\frac{\partial h_i}{\partial o_i} = \frac{\partial \tanh(o_i)}{\partial o_i} = 1 - \tanh(o_i)^2$$

$$o_i = \sum_{j=1}^{kd} w_{ij} e_j + b_i$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial h_i} = \sum_{s=1}^{|V|} (\hat{y}_s - \mathbb{I}(n_u = s)) \times u_{si}$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial b_i} = \frac{\partial -\log \hat{y}_{n_u}}{\partial h_i} \times \frac{\partial h_i}{\partial o_i} \times \frac{\partial o_i}{\partial b_i}$$

$$= \left(\sum_{s=1}^{|V|} (\hat{y}_s - \mathbb{I}(n_u = s)) \times u_{si} \right) \times (1 - h_i^2)$$

$$\mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_{d_h} \end{bmatrix} \in \mathbb{R}^{d_h \times 1}$$

$$\mathbf{h} = \begin{bmatrix} h_1 \\ \vdots \\ h_{d_h} \end{bmatrix} \in \mathbb{R}^{d_h \times 1}$$

$$\mathbf{e} = \begin{bmatrix} e_1 \\ \vdots \\ e_{kd} \end{bmatrix} \in \mathbb{R}^{kd \times 1}$$

$$W = \begin{bmatrix} w_{11} & \cdots & w_{1kd} \\ \vdots & \ddots & \vdots \\ w_{d_h 1} & \cdots & w_{d_h kd} \end{bmatrix} \in \mathbb{R}^{d_h \times kd}$$

$$\mathbf{h} = \tanh(W\mathbf{e} + \mathbf{b}) \quad o_i = \sum_{j=1}^{kd} w_{ij}e_j + b_i$$

$$\nabla_{\mathbf{e}}(-\log \hat{y}_{n_u}) = \begin{bmatrix} \frac{\partial -\log \hat{y}_{n_u}}{\partial e_1} \\ \vdots \\ \frac{\partial -\log \hat{y}_{n_u}}{\partial e_{kd}} \end{bmatrix}$$

$$h_i = \tanh(o_i) = \frac{\exp(o_i) - \exp(-o_i)}{\exp(o_i) + \exp(-o_i)}$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial e_s} = \sum_{i=1}^{d_h} \left(\frac{\partial -\log \hat{y}_{n_u}}{\partial h_i} \times \frac{\partial h_i}{\partial o_i} \times \frac{\partial o_i}{\partial e_s} \right)$$

$$h_i = \tanh(o_i) = \frac{\exp(o_i) - \exp(-o_i)}{\exp(o_i) + \exp(-o_i)}$$

$$\frac{\partial \tanh(o_i)}{\partial o_i} = 1 - \tanh(o_i)^2$$

$$o_i = \sum_{j=1}^{kd} w_{ij} e_j + b_i$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial h_i} = \sum_{s=1}^{|V|} (\hat{y}_s - \mathbb{I}(n_u = s)) \times u_{si}$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial e_s} = \sum_{i=1}^{d_h} \left(\frac{\partial -\log \hat{y}_{n_u}}{\partial h_i} \times \frac{\partial h_i}{\partial o_i} \times \frac{\partial o_i}{\partial e_s} \right)$$

$$= \sum_{i=1}^{d_h} \left(\sum_{s=1}^{|V|} (\hat{y}_s - \mathbb{I}(n_u = s)) \times u_{si} \times (1 - h_i^2) \times w_{is} \right)$$

$$\mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_{d_h} \end{bmatrix} \in \mathbb{R}^{d_h \times 1}$$

$$\mathbf{h} = \begin{bmatrix} h_1 \\ \vdots \\ h_{d_h} \end{bmatrix} \in \mathbb{R}^{d_h \times 1}$$

$$\mathbf{e} = \begin{bmatrix} e_1 \\ \vdots \\ e_{kd} \end{bmatrix} \in \mathbb{R}^{kd \times 1}$$

$$\mathbf{W} = \begin{bmatrix} w_{11} & \cdots & w_{1kd} \\ \vdots & \ddots & \vdots \\ w_{d_h 1} & \cdots & w_{d_h kd} \end{bmatrix} \in \mathbb{R}^{d_h \times kd}$$

$$\mathbf{h} = \tanh(\mathbf{W}\mathbf{e} + \mathbf{b})$$

$$o_i = \sum_{j=1}^{kd} w_{ij} e_j + b_i$$

$$\nabla_{\mathbf{W}}(-\log \hat{y}_{n_u}) = \begin{bmatrix} \frac{\partial -\log \hat{y}_{n_u}}{\partial w_{11}} & \cdots & \frac{\partial -\log \hat{y}_{n_u}}{\partial w_{1kd}} \\ \vdots & \ddots & \vdots \\ \frac{\partial -\log \hat{y}_{n_u}}{\partial w_{d_h 1}} & \cdots & \frac{\partial -\log \hat{y}_{n_u}}{\partial w_{d_h kd}} \end{bmatrix}$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial w_{lm}} \text{ ?}$$

$$h_i = \tanh(o_i) = \frac{\exp(o_i) - \exp(-o_i)}{\exp(o_i) + \exp(-o_i)}$$

$$\frac{\partial \tanh(o_i)}{\partial o_i} = 1 - \tanh(o_i)^2$$

$$o_i = \sum_{j=1}^{kd} w_{ij} e_j + b_i$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial h_i} = \sum_{s=1}^{|V|} (\hat{y}_s - \mathbb{I}(n_u = s)) \times u_{si}$$

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial w_{lm}} = \frac{\partial -\log \hat{y}_{n_u}}{\partial h_l} \times \frac{\partial h_l}{\partial o_l} \times \frac{\partial o_l}{\partial w_{lm}}$$

$$= \left(\sum_{s=1}^{|V|} (\hat{y}_s - \mathbb{I}(n_u = s)) \times u_{sl} \right) \times (1 - h_i^2) \times e_m$$

$$E = \begin{bmatrix} E_{11} & \cdots & E_{1|V|} \\ \vdots & \ddots & \vdots \\ E_{d1} & \cdots & E_{d|V|} \end{bmatrix} \in \mathbb{R}^{d \times |V|} \quad \mathbf{e} = [Et_{n1}; Et_{n2}; \dots; Et_{nk}] = \begin{bmatrix} e_1 \\ \vdots \\ e_{kd} \end{bmatrix}$$

$$Et_{ni} = \begin{bmatrix} E_{11} & \cdots & E_{1|V|} \\ \vdots & \ddots & \vdots \\ E_{d1} & \cdots & E_{d|V|} \end{bmatrix} \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix} \longrightarrow e_s \in \mathbf{e} (1 \leq s \leq kd) \text{ 是矩阵 } E \text{ 中的某个元素}$$

因此

$$\frac{\partial -\log \hat{y}_{n_u}}{\partial E_{lm}} = \begin{cases} \frac{\partial -\log \hat{y}_{n_u}}{\partial e_s} & , E_{lm} \text{ 是 } \mathbf{e} \text{ 中的元素, 记 } E_{lm} \triangleq e_s \\ 0 & , E_{lm} \text{ 不是 } \mathbf{e} \text{ 中的元素} \end{cases}$$

- 神经语言模型
 - ✓背景
 - ✓前馈神经语言模型
 - ✓模型训练
- 词嵌入
 - ✓背景
 - ✓向量语义
 - ✓Word2vec
- 讨论

自然语言理解：研究计算机理解和运用人类自然语言（如汉语、英语等）的理论方法和技术，实现人机之间的自然语言交流



北京大学创办于1898年，初名京师大学堂，是中国第一所国立综合性大学，也是当时中国最高教育行政机关。辛亥革命后，于1912年改为现名。

作为新文化运动的中心和“五四”运动的策源地，作为中国最早传播马克思主义和民主科学思想的发祥地，作为中国共产党最早的活动基地，北京大学为民族的振兴和解放、国家的建设和发展、社会的文明和进步做出了不可替代的贡献，在中国走向现代化的进程中起到了重要的先锋作用。爱国、进步、民主、科学的传统精神和勤奋、严谨、求实、创新的学风在这里生生不息、代代相传。

1917年，著名教育家蔡元培出任北京大学校长，他“循思想自由原则，取兼容并包主义”，对北京大学进行了卓有成效的改革，促进了思想解放和学术繁荣。陈独秀、李大钊、毛泽东以及鲁迅、胡适等一批杰出人才都曾在北京大学任职或任教。

1937年卢沟桥事变后，北京大学与清华大学、南开大学南迁长沙，共同组成长沙临时大学。不久，临时大学又迁到昆明，改称国立西南联合大学。抗日战争胜利后，北京大学于1946年10月在北平复学。

如何理解并表示文本的语义，特别是词的意思是自然语言理解及处理的基础

前馈神经语言模型采用了向量的形式来表示词。本节课系统阐述以向量的形式表示词的历史和相关技术

- 词的意思
 - ✓具有多对多的特性（一词多意，多词一意）
- 多种关系
 - ✓同义
 - ✓反义
 - ✓相关
 - ✓隐含

- 为了能有效理解自然语言，我们能否构建一套表示词义的理论，这套理论至少能支持部分人们对词义的理解
- 显然，以字符串来表达词及其语义的方法是行不通的
 - 不够灵活，循环定义
- 向量语义 (vector semantics)
 - 自然语言处理的词义表示标准模型
 - 也是深度学习技术能用于自然语言理解和处理的基础

- 思想1: Defining meaning by linguistic distribution
 - ✓ 维特根斯坦 (Ludwig Wittgenstein)
 - The meaning of a word is its use in the language
 - One way to define “usage” : words are defined by their environments (the words around them)
 - ✓ 哈里斯 (Zellig Harris)
 - If A and B have almost identical environments we say that they are synonyms
- 词义: the meaning of a word by its distribution in language use, meaning its neighboring words or grammatical environments
 - 分布语义 (Distributional semantics) : 词的意思 (语义) 由其上下文 (context) 来定义
 - 想想看, 我们是不是这样理解陌生词的词义

- 思想2: Meaning as a point in multidimensional space
 - ✓ PAD情感三维理论: 情感具有P (愉悦度) 、A (激活度) 和D (优势度) 三个维度, 情感语义可以用这三个维度上的值构成的向量表示或度量
 - P: pleasantness
 - A: arousal
 - D: dominance

	Word	Score		Word	Score
Valence	love	1.000		toxic	0.008
	happy	1.000		nightmare	0.005
Arousal	elated	0.960		mellow	0.069
	frenzy	0.965		napping	0.046
Dominance	powerful	0.991		weak	0.045
	leadership	0.983		empty	0.081

- 源自
 - ✓ Defining meaning by linguistic distribution
 - ✓ Meaning as a point in multidimensional space



- 用向量来表示词的意思，称为词嵌入 (word embedding)
 - ✓把词嵌入到语义空间
 - ✓狭义上的词嵌入指通过神经网络/深度学习学到的稠密向量，如word2vec
 - ✓当前（今后），自然语言处理算法都用词嵌入来表示词的意思
- 为什么用向量来表示词的语义
 - ✓如果用字符串（词本身）
 - 在训练和测试的时候，要求一样的词
 - ✓如果用向量
 - 通过向量的相似性，可以泛化到类似的词
 - 可以把各类自然语言处理任务变成向量计算，非常灵活和高效

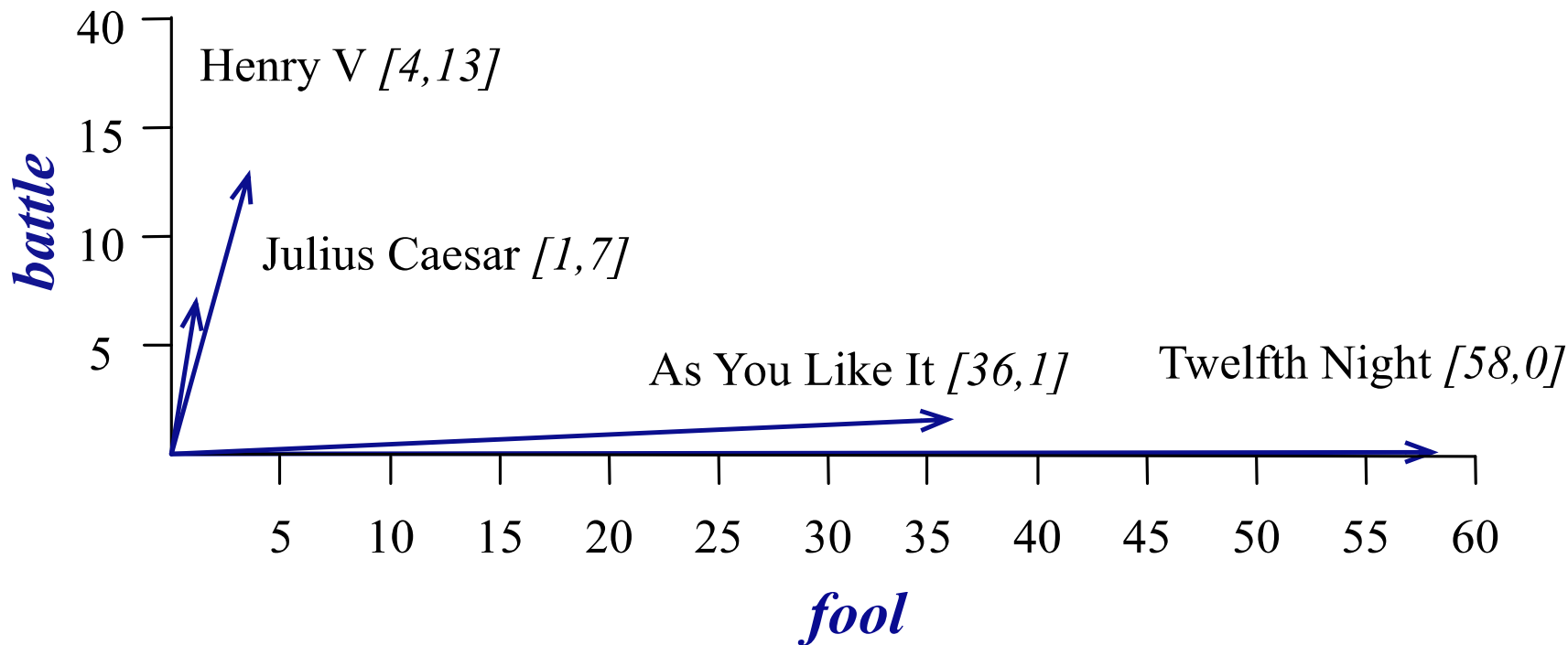
- 用词的向量语义表示，而非词的字符串表示来开展自然语言理解和应用工作
 - ✓ 荃者所以在鱼，得鱼而忘荃
 - Nets are for fish
 - Once you get the fish, you can forget the net
 - ✓ 言者所以在意，得意而忘言
 - Words are for meaning
 - Once you get the meaning, you can forget the words

- 共现矩阵
 - ✓ co-occurrence matrix, 也称共生矩阵
 - ✓ 表示词之间共同出现的情况
 - ✓ 词向量模型本质上都基于共现矩阵
- 典型的共现矩阵
 - ✓ Term-document matrix
 - ✓ Term-term matrix (or word-word matrix)

以词为维度，每篇文档表示成词空间中的一个向量

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

文档向量可视化



文档向量是信息检索的基础

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

两个喜剧的向量比较相似
两个历史剧的向量比较相似

词向量：文档维度

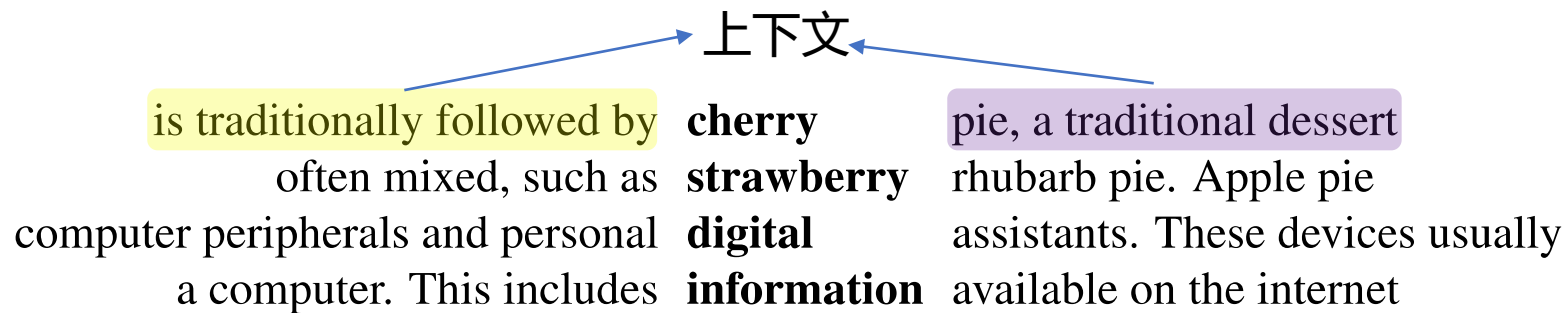
	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

battle is "the kind of word that occurs in Julius Caesar and Henry V"

fool is "the kind of word that occurs in comedies, especially Twelfth Night"

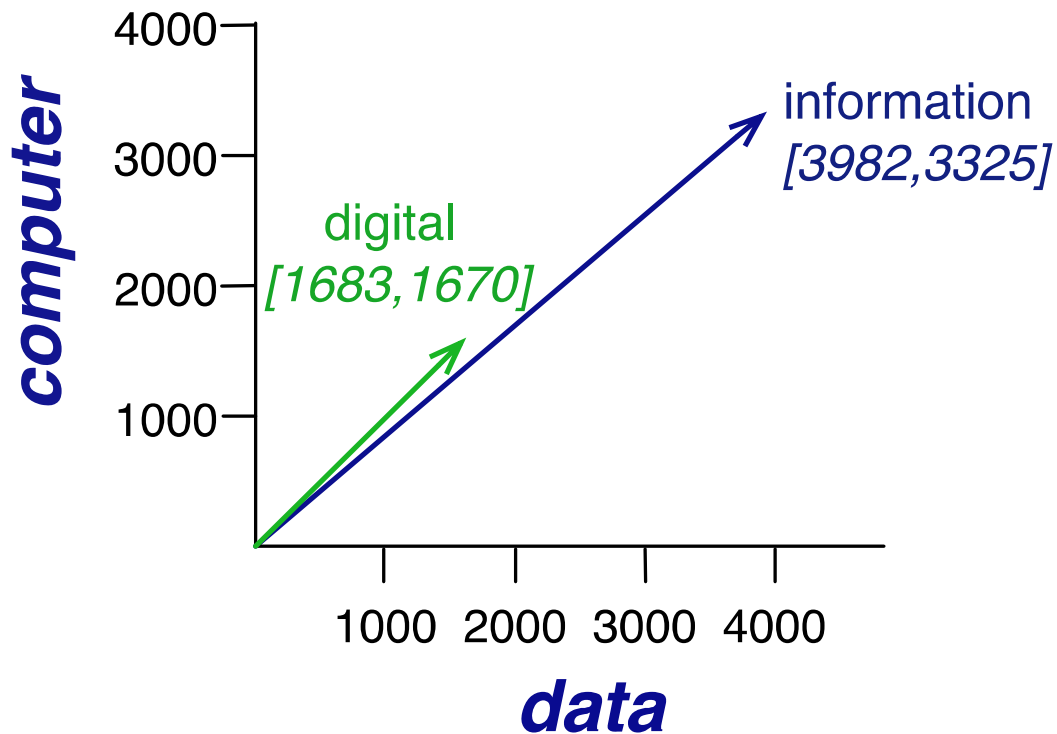
词向量：词维度

如果两个词的上下文向量是相似的，那么这两个词的意思是相近的



	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

词向量可视化



- 到目前未知，我们知道词可以用向量表示，这也是所有词嵌入方法的基本思想
- 那么，如何通过词向量来计算（度量）两个词的相似性呢？
 - ✓ 向量空间
 - 向量内积 (dot product)
 - 向量夹角余弦(cosine)

- 定义
$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^N v_i w_i = v_1 w_1 + v_2 w_2 + \dots + v_N w_N$$

- 不足

- ✓ 向量内积有利于长向量

- 如果一个向量很长，而且在很多维度上值比较大时，内积会比较大

- 向量长度（模）
$$|\mathbf{v}| = \sqrt{\sum_{i=1}^N v_i^2}$$

- 常用词（停用词，如冠词、介词、人称代词等）的向量长度很长，内积会偏好这些词。这显然有问题，因为两个词的相似性应该与它们出现的频率无关

- 定义

$$\text{cosine}(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

- 基于向量内积

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

$$\frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}| |\mathbf{b}|} = \cos \theta$$

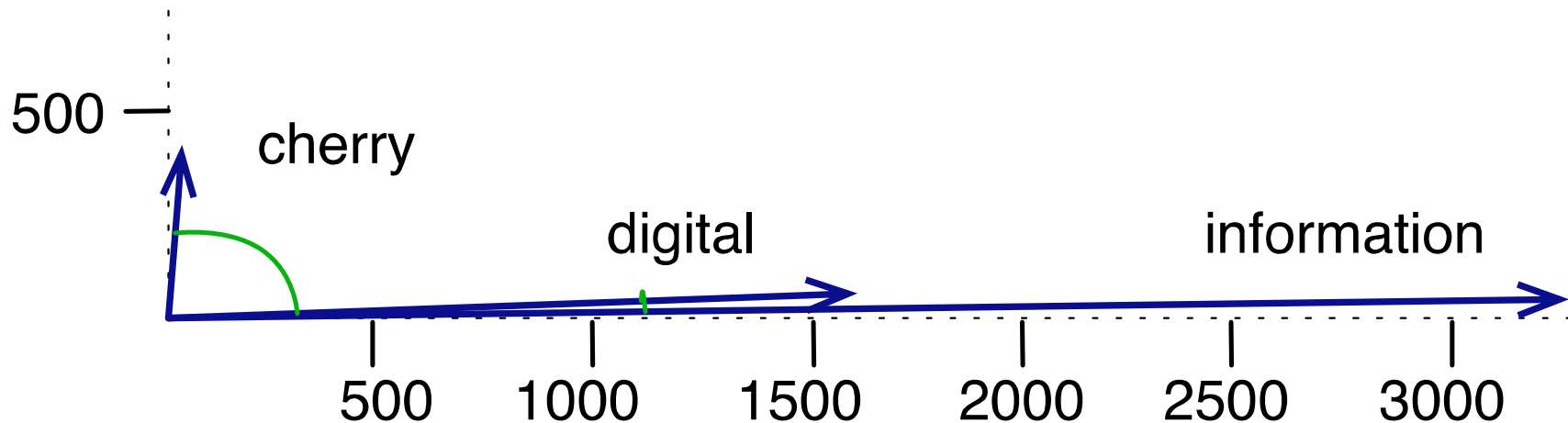
$$\cos(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}| |\vec{w}|} = \frac{\vec{v}}{|\vec{v}|} \cdot \frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^N v_i w_i}{\sqrt{\sum_{i=1}^N v_i^2} \sqrt{\sum_{i=1}^N w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

$$\cos(\text{cherry}, \text{information}) = \frac{442 * 5 + 8 * 3982 + 2 * 3325}{\sqrt{442^2 + 8^2 + 2^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .017$$

$$\cos(\text{digital}, \text{information}) = \frac{5 * 5 + 1683 * 3982 + 1670 * 3325}{\sqrt{5^2 + 1683^2 + 1670^2} \sqrt{5^2 + 3982^2 + 3325^2}} = .996$$

Dimension 1: 'pie'



Dimension 2: 'computer'

- 仅仅考虑词频
- 尽管词频提供了有用的信息，但仅仅依靠词频是不够的
 - ✓ 过于频繁出现的词，如冠词、代词等提供的上下文信息并不多
- 这就存在一个矛盾，如何有效计算共现矩阵中词的权重
 - ✓ 两个常用的方法

- Tf-idf

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

- PMI (Pointwise mutual information)

$$\text{PMI}(w_1, w_2) = \log \frac{p(w_1, w_2)}{p(w_1)p(w_2)}$$

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

- Tf (term frequency)

- ✓ $\text{tf}_{t,d} = \text{count}(t,d)$

- ✓ $\text{tf}_{t,d} = \log_{10}(\text{count}(t,d) + 1)$

注意：这里文档可以是任何大小的文本，例如：小说、网页、新闻、甚至是一个段落

- Df (Document frequency)

- ✓ 词在多少篇文档中出现

- 例如：“Romeo” 在莎士比亚剧作中是非常特殊的一个词

	Collection Frequency	Document Frequency
Romeo	113	1
action	113	31

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

- Idf (inverse document frequency)

$$\text{idf}_t = \log \left(\frac{N}{\text{df}_t} \right)$$

$$\text{idf}_t = \log \left(1 + \frac{N}{\text{df}_t} \right)$$

✓ N 是文档集中文档数量

Word	df	idf
Romeo	1	1.57
salad	2	1.27
Falstaff	4	0.967
forest	12	0.489
battle	21	0.246
wit	34	0.037
fool	36	0.012
good	37	0
sweet	37	0

$$w_{t,d} = \text{tf}_{t,d} \times \text{idf}_t$$

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	0.074	0	0.22	0.28
good	0	0	0	0
fool	0.019	0.021	0.0036	0.0083
wit	0.049	0.044	0.018	0.022

- Pointwise Mutual Information (点互信息)

✓来衡量两个事件关联强度的统计量。值越大，表明两个事件的关联性越强，值越小表明两个事件关联性越弱

$$\text{PMI}(X, Y) = \log_2 \frac{P(x, y)}{P(x)P(y)}$$

- 两个词间的PMI

$$\text{PMI}(\text{word}_1, \text{word}_2) = \log_2 \frac{P(\text{word}_1, \text{word}_2)}{P(\text{word}_1)P(\text{word}_2)}$$

- PMI值域为 $(-\infty, +\infty)$
- PMI取负值存在问题
 - ✓取负值，意味着两个事件同时出现的可能性比随机情况下（独立）两者同时出现的概率还要低，这个理解上有点麻烦
 - ✓负值表示负相关，大家对“负相关”之了解和关心远低于“正相关”
- 因此，把负值的PMI替换成0，就变成PPMI
- 两个词间的PPMI

$$\text{PPMI}(\text{word}_1, \text{word}_2) = \max\left(\log_2 \frac{P(\text{word}_1, \text{word}_2)}{P(\text{word}_1)P(\text{word}_2)}, 0\right)$$

- Term-term Matrix F: W 行(词, words) and C 列 (上下文, contexts)
- 定义 f_{ij} 为词 w_i 出现在上下文 c_j 的次数

$$p_{ij} = \frac{f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

$$p_{i*} = \frac{\sum_{j=1}^C f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$

$$p_{*j} = \frac{\sum_{i=1}^W f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$

$$pmi_{ij} = \log_2 \frac{p_{ij}}{p_{i*} p_{*j}}$$

$$ppmi_{ij} = \begin{cases} pmi_{ij} & \text{if } pmi_{ij} > 0 \\ 0 & \text{otherwise} \end{cases}$$

$$p_{ij} = \frac{f_{ij}}{\sum_{i=1}^W \sum_{j=1}^C f_{ij}}$$

	computer	data	result	pie	sugar	count(w)
cherry	2	8	9	442	25	486
strawberry	0	0	1	60	19	80
digital	1670	1683	85	5	4	3447
information	3325	3982	378	5	13	7703
count(context)	4997	5673	473	512	61	11716

$$p(w=\text{information}, c=\text{data}) = 3982/111716 = .3399$$

$$p(w=\text{information}) = 7703/111716 = .6575$$

$$p(c=\text{data}) = 5673/111716 = .4842$$

$$p(w_i) = \frac{\sum_{j=1}^C f_{ij}}{N} \quad p(c_j) = \frac{\sum_{i=1}^W f_{ij}}{N}$$

	p(w,context)					p(w)
	computer	data	result	pie	sugar	p(w)
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	0.3399	0.0323	0.0004	0.0011	0.6575
p(context)	0.4265	0.4842	0.0404	0.0437	0.0052	

$$pmi_{ij} = \log_2 \frac{p_{ij}}{p_{i*} p_{*j}}$$

	p(w,context)					p(w)
	computer	data	result	pie	sugar	p(w)
cherry	0.0002	0.0007	0.0008	0.0377	0.0021	0.0415
strawberry	0.0000	0.0000	0.0001	0.0051	0.0016	0.0068
digital	0.1425	0.1436	0.0073	0.0004	0.0003	0.2942
information	0.2838	0.3399	0.0323	0.0004	0.0011	0.6575
p(context)	0.4265	0.4842	0.0404	0.0437	0.0052	

$$pmi(\text{information}, \text{data}) = \log_2 (.3399 / (.6575 * .4842)) = .0944$$

PPMI matrix

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

- PMI偏向不频繁事件

✓低频词可能有很大的PMI值

- 解决方案

✓调节低频词的概率

– 提升上下文词的概率值

– $\alpha = 0.75$

✓采用类似之前拉普拉斯平滑的方法

$$\text{PPMI}_{\alpha}(w, c) = \max(\log_2 \frac{P(w, c)}{P(w)P_{\alpha}(c)}, 0)$$

$$P_{\alpha}(c) = \frac{\text{count}(c)^{\alpha}}{\sum_c \text{count}(c)^{\alpha}}$$

考虑 $P(a) = .99$ and $P(b) = .01$

$$P_{\alpha}(a) = \frac{.99^{.75}}{.99^{.75} + .01^{.75}} = .97 \quad P_{\alpha}(b) = \frac{.01^{.75}}{.01^{.75} + .01^{.75}} = .03$$

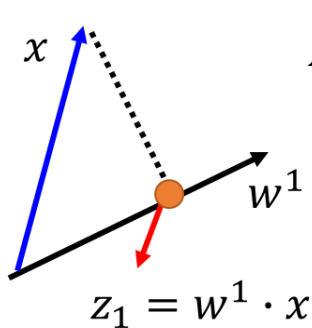
$$P_{\alpha}(b) > P(b)$$

- tf-idf (or PMI) 向量
 - ✓高维（维度为词汇量 $|V|$ ，通常 $|V|= 20,000$ to $50,000$ ）
 - ✓稀疏（很多维度上的值为0）
- 替代方案
 - ✓学习词语义表示向量，具备
 - 低维（维度50-1000）
 - 稠密（大多数维度上的值不为0）

- 为什么要用低维稠密向量
 - ✓ 在机器学习中更好用于训练模型
 - ✓ 泛化性能更好
 - ✓ 更能捕获一些词义关系，如同义词
 - ✓ 最最重要的：在实际应用中， they work better

- 主成分分析 (Principal Component Analysis, PCA)
- 奇异值分解 (Singular Value Decomposition, SVD)
- 词嵌入 (word embedding)
 - ✓ 神经语言模型启发的方法, 如Word2vec (skipgram, CBOW), GloVe
- 动态方法
 - ✓ Contextual Embeddings (Transformer及基于其的大模型等)

基本思想：从原始变量中导出少数几个主成分，使它们尽可能多地保留原始变量的信息（用方差来衡量）



$$\vec{X} \rightarrow [z_1, z_2, \dots, z_k]^T \quad z_i = \vec{w}^i \cdot \vec{X}$$

$$w^i \cdot w^j = 0; i \neq j$$

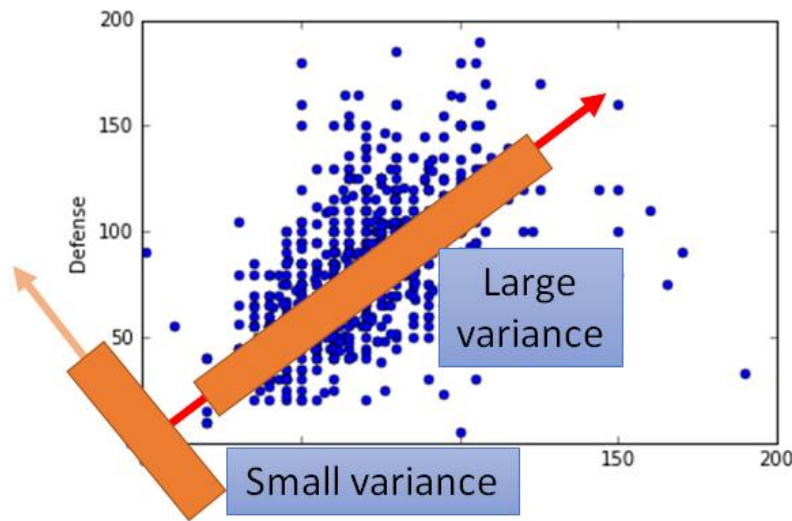
$$z_1 = \vec{w}^1 \cdot \vec{X}$$

求解

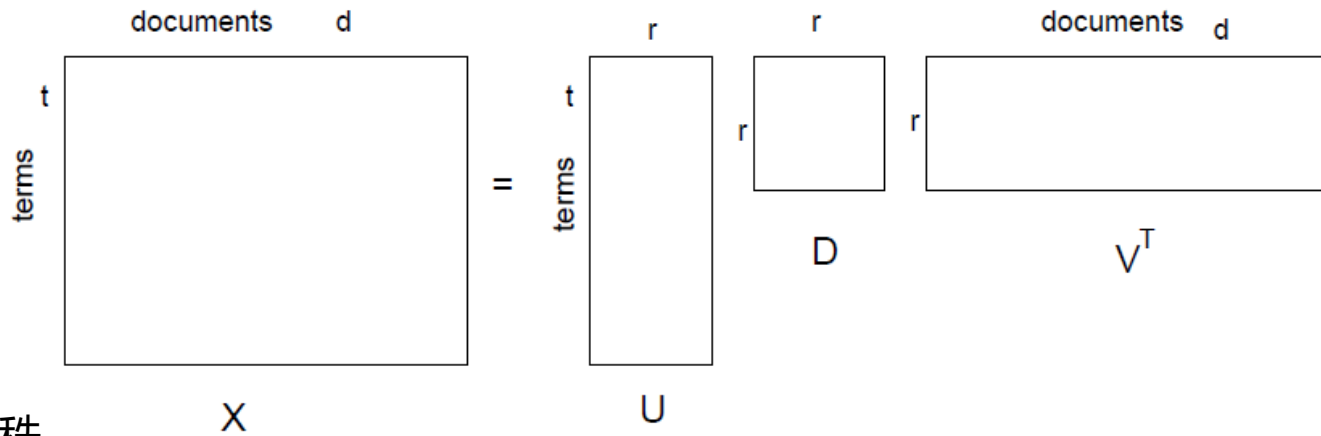
$$\max_{w^1} \text{Var}(z_1) = \frac{1}{N} \sum_{z_1} (z_1 - \bar{z}_1)^2$$

约束条件

$$\|w^1\|_2 = 1$$



Singular Value Decomposition (SVD)



r : 矩阵 X 的秩

t : 词总数

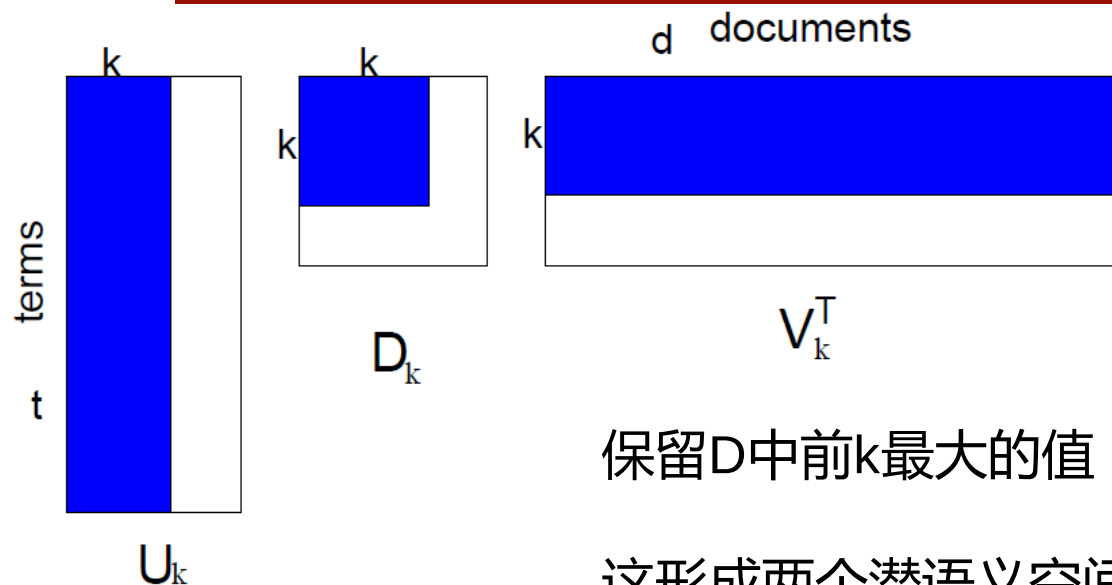
d : 文档数

D : 对角方矩, 对角线元素为奇异值, 降序排列

奇异值是 XX^T (X^TX) 特征值的平方根

U : X 的左奇异向量构成 (XX^T 的特征向量)

V : X 的右奇异向量构成 (X^TX 的特征向量)



保留 D 中前 k 最大的值（最主要成分）形成矩阵 D_k

这形成两个潜语义空间 (latent semantic spaces)

- ✓ $U_k D_k$ 表示主题/概念空间中词的潜语义向量
- ✓ V_k 表示主题/概念空间中文档的潜语义向量

	d1	d2	d3	d4	d5	d6
gnarly	1	0	1	0	0	0
wicked	0	1	0	1	0	0
awesome	1	1	1	1	0	0
lame	0	0	0	0	1	1
terrible	0	0	0	0	0	1

Distance from *gnarly*

1. gnarly
2. awesome
3. terrible
4. wicked
5. lame

	T(erm)				
gnarly	0.41	0.00	0.71	0.00	-0.58
wicked	0.41	0.00	-0.71	0.00	-0.58
awesome	0.82	-0.00	-0.00	-0.00	0.58
lame	0.00	0.85	0.00	-0.53	0.00
terrible	0.00	0.53	0.00	0.85	0.00

	S(ingular values)				
	2.45	0.00	0.00	0.00	0.00
	0.00	1.62	0.00	0.00	0.00
	0.00	0.00	1.41	0.00	0.00
	0.00	0.00	0.00	0.62	0.00
	0.00	0.00	0.00	0.00	-0.00

	D(ocument)				
d1	0.50	-0.00	0.50	0.00	-0.71
d2	0.50	0.00	-0.50	0.00	0.00
d3	0.50	-0.00	0.50	0.00	0.71
d4	0.50	-0.00	-0.50	-0.00	0.00
d5	-0.00	0.53	0.00	-0.85	0.00
d6	0.00	0.85	0.00	0.53	0.00

 T

gnarly	0.41	0.00
wicked	0.41	0.00
awesome	0.82	-0.00
lame	0.00	0.85
terrible	0.00	0.53

 $\times$

2.45	0.00
0.00	1.62

 $=$

gnarly	1.00	0.00
wicked	1.00	0.00
awesome	2.00	0.00
lame	0.00	1.38
terrible	0.00	0.85

Distance from *gnarly*

1. gnarly
2. wicked
3. awesome
4. terrible
5. lame

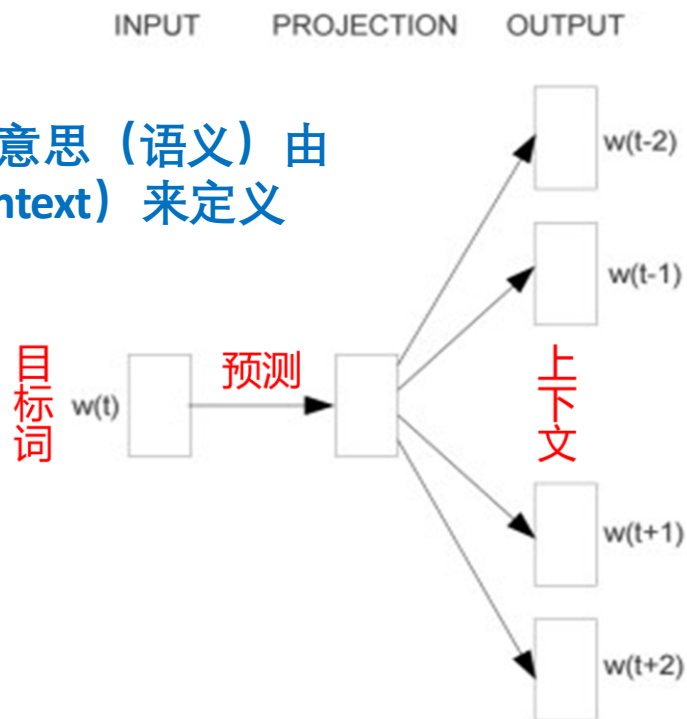
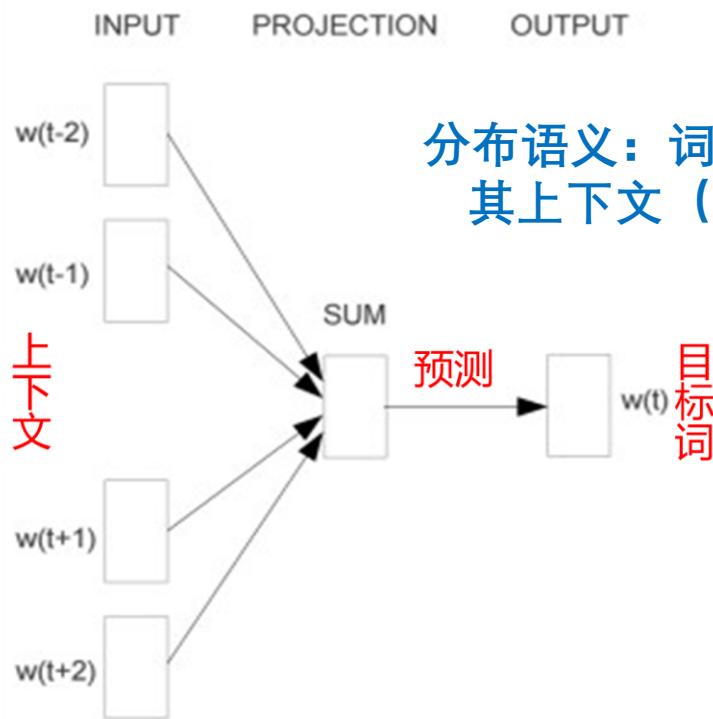
- 以上方法都是从数学模型的角度对词向量进行降维
- 词嵌入从学习的角度来直接生成低维稠密词向量
 - ✓ 这类方法基于词分布语义假说，采用神经网络对词向量进行建模，直接学习词向量。其核心是上下文的表示以及上下文与目标词之间的关系建模。Word2Vec是这类方法的代表
- 词嵌入实际上是神经语言模型的副产品

- 词嵌入代表性模型
- 简单，训练快
- 思想
 - ✓ 通过学习一个分类器对词进行预测而非词频计算词向量
 - ✓ 虽然模型是有监督的分类模型，但通过自监督（self-supervision）的方式，不需要对数据进行标注
 - ✓ 自监督学习是大规模预训练模型的主要学习方法，也是预训练大模型成功的关键

C-BOW模型

<

Skip-Gram模型



- 假定目标词上下文窗口为2，训练数据例子如下

...lemon, a [tablespoon of apricot jam, a] pinch...

- 目标
 - ✓ 给定候选词对，训练一个分类器，给出词对中的词是否是上下文的概率

(apricot, jam)

$$P(+ | w, c)$$

(apricot, aardvark)

$$P(- | w, c) = 1 - P(+ | w, c)$$

...

- 如何计算概率
- 回顾：如果两个向量内积大，则表明这两个向量相似
 - ✓ 因此, $\text{Similarity}(w, c) \propto w \cdot c$
- 内积不是概率，需要进行转换
 - ✓ 利用sigmoid函数可达到目的

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

$$P(-|w, c) = 1 - P(+|w, c) = \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)}$$

- 对一个上下文词，其概率计算如下

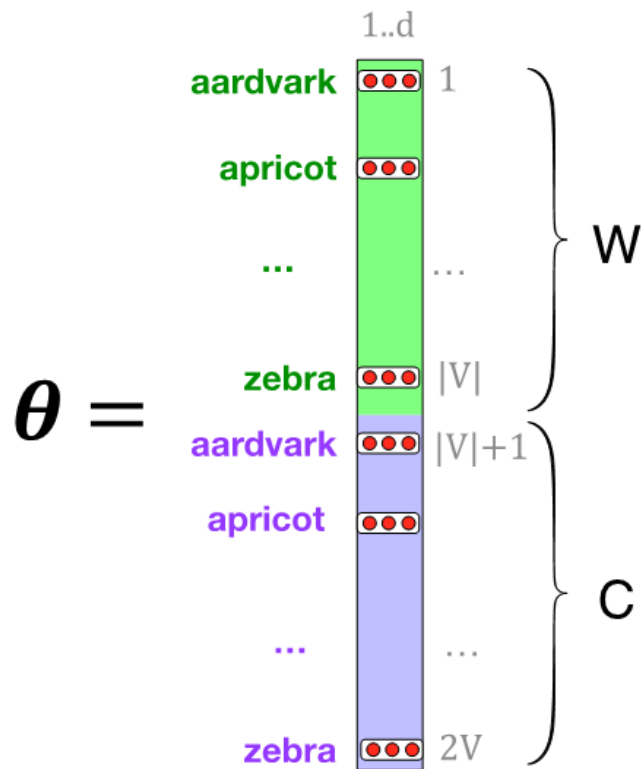
$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

- 对多个上下文词，可假定其相互独立，因此

$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

- 训练一个概率分类器，给定
 - ✓ 目标词 w
 - ✓ 目标词 w 的上下文窗口 $c_{1:L}$ (L 个词)
- 基于目标词 w 和上下文窗口 $c_{1:L}$ 中各词，可以计算这些词是 w 的上下文的概率
 - ✓ 要计算以上概率，只需要有所有词的词向量
 - ✓ 通过训练一个简单的前馈神经网络，就可以学习出所有词的词向量，并生成一个预测目标词上下文词的分类器



作为目标词的词向量

注意：每个词即可作为目标词，也可以是上下文词，因此有两个词嵌入向量

作为上下文词的词向量

...lemon, a [tablespoon of apricot jam, a] pinch...

positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

学习分类器需要负样本。为此，通过词频对词汇表进行采样，为每个正样本生成 k 个负样本
注意要约束负样本中的词不是目标词的上下文词

negative examples -

t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

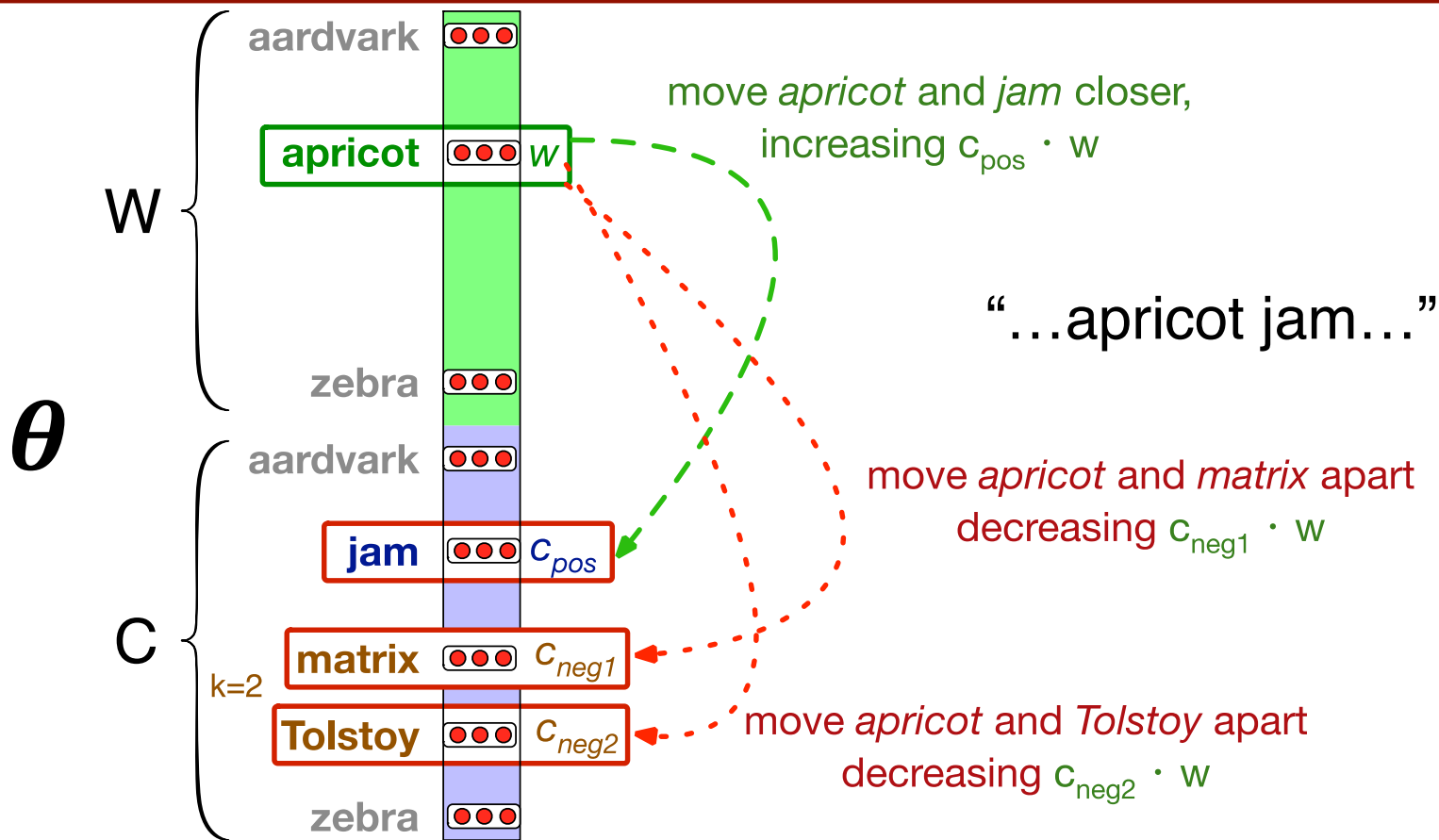
$k = 2$

- 给定训练集（包含正负样本）以及嵌入向量的初始值
- 学习（训练）的目标是调整嵌入向量，使得
 - ✓ 最大化来自正样本 (w, c_{pos}) 上下文词 c_{pos} 出现的概率 $P(+|w, c_{pos}) = P(c_{pos}|w)$
 - ✓ 最小化来自负样本 (w, c_{neg}) 上下文词 c_{neg} 出现的概率 $P(+|w, c_{neg}) = P(c_{neg}|w)$
 - 等价于最大化来自负样本 (w, c_{neg}) 上下文词 c_{neg} 不出现的概率 $P(-|w, c_{neg}) = 1 - P(c_{neg}|w) = 1 - P(+|w, c_{neg})$

- 对 w 和 $c_{pos}, c_{neg1} \dots c_{negk}$ 目标函数为

$$\begin{aligned} L_{CE} &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\ \min \quad &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\ &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned}$$

- 如何学习参数 W (嵌入向量)
 - ✓ 梯度下降!
- 利用梯度下降方法, 修改嵌入向量, 来最小化目标函数
 - ✓ 正样本更有可能出现
 - ✓ 负样本尽可能不出现



$$\frac{\partial L_{CE}}{\partial c_{pos}} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{w}$$

$$\frac{\partial L_{CE}}{\partial c_{neg}} = [\sigma(\mathbf{c}_{neg} \cdot \mathbf{w})] \mathbf{w}$$

$$\frac{\partial L_{CE}}{\partial \mathbf{w}} = [\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}) - 1] \mathbf{c}_{pos} + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w})] \mathbf{c}_{neg_i}$$

$$L_{CE} = - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right]$$

$$\mathbf{c}_{pos}^{t+1} = \mathbf{c}_{pos}^t - \eta [\sigma(\mathbf{c}_{pos}^t \cdot \mathbf{w}^t) - 1] \mathbf{w}^t$$

$$\mathbf{c}_{neg}^{t+1} = \mathbf{c}_{neg}^t - \eta [\sigma(\mathbf{c}_{neg}^t \cdot \mathbf{w}^t)] \mathbf{w}^t$$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \eta \left[[\sigma(\mathbf{c}_{pos} \cdot \mathbf{w}^t) - 1] \mathbf{c}_{pos} + \sum_{i=1}^k [\sigma(\mathbf{c}_{neg_i} \cdot \mathbf{w}^t)] \mathbf{c}_{neg_i} \right]$$

- 对每个词Skip-Gram模型学习出两类向量
 - ✓ 目标嵌入向量 w
 - ✓ 上下文嵌入向量 c
- 词的向量如何表示
 - ✓ $w + c$

- GloVe

- ✓关键思想：考虑目标词和上下文词的共现频次

- ✓训练快，可扩展性好，能应用于大语料库

$$J(\theta) = \sum_{i,j \in V} f(X_{i,j}) \left(\mathbf{u}_i \cdot \mathbf{v}_j + b_i + \tilde{b}_j - \log X_{i,j} \right)^2$$

- FastText

- ✓子词嵌入 (Subword Embeddings)

- ✓把每个词分解成n-grams ($3 \leq n \leq 6$)

- ✓对字词做嵌入

where: 3-grams: <wh, whe, her, ere, re>

4-grams: <whe, wher, here, ere>

5-grams: <wher, where, here>

6-grams: <where, where>

expect =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$



词聚集性



单词类比

a:b :: c:?



$$d = \arg \max_x \frac{(w_b - w_a + w_c)^T w_x}{||w_b - w_a + w_c||}$$

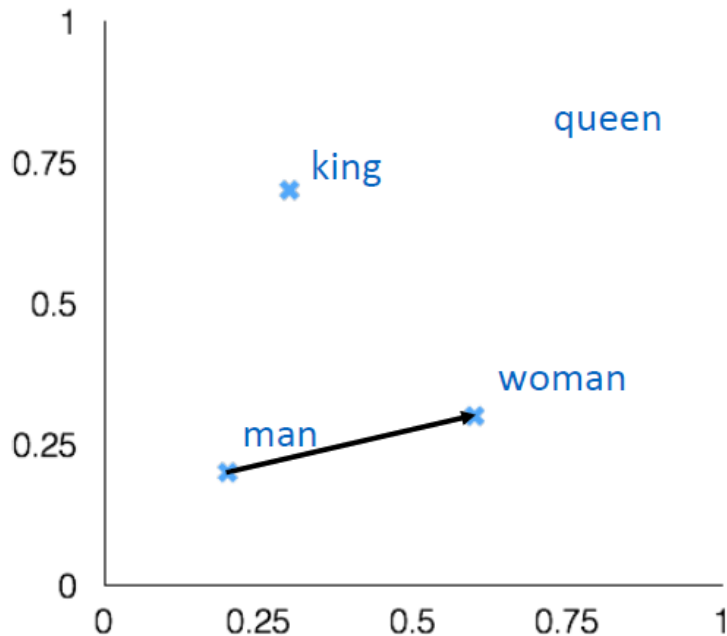
man:woman :: king:?

+ king [0.30 0.70]

- man [0.20 0.20]

+ woman [0.60 0.30]

queen [0.70 0.80]



关系捕获

(性别) $\theta_{\text{king}} - \theta_{\text{man}} \approx \theta_{\text{queen}} - \theta_{\text{woman}}$

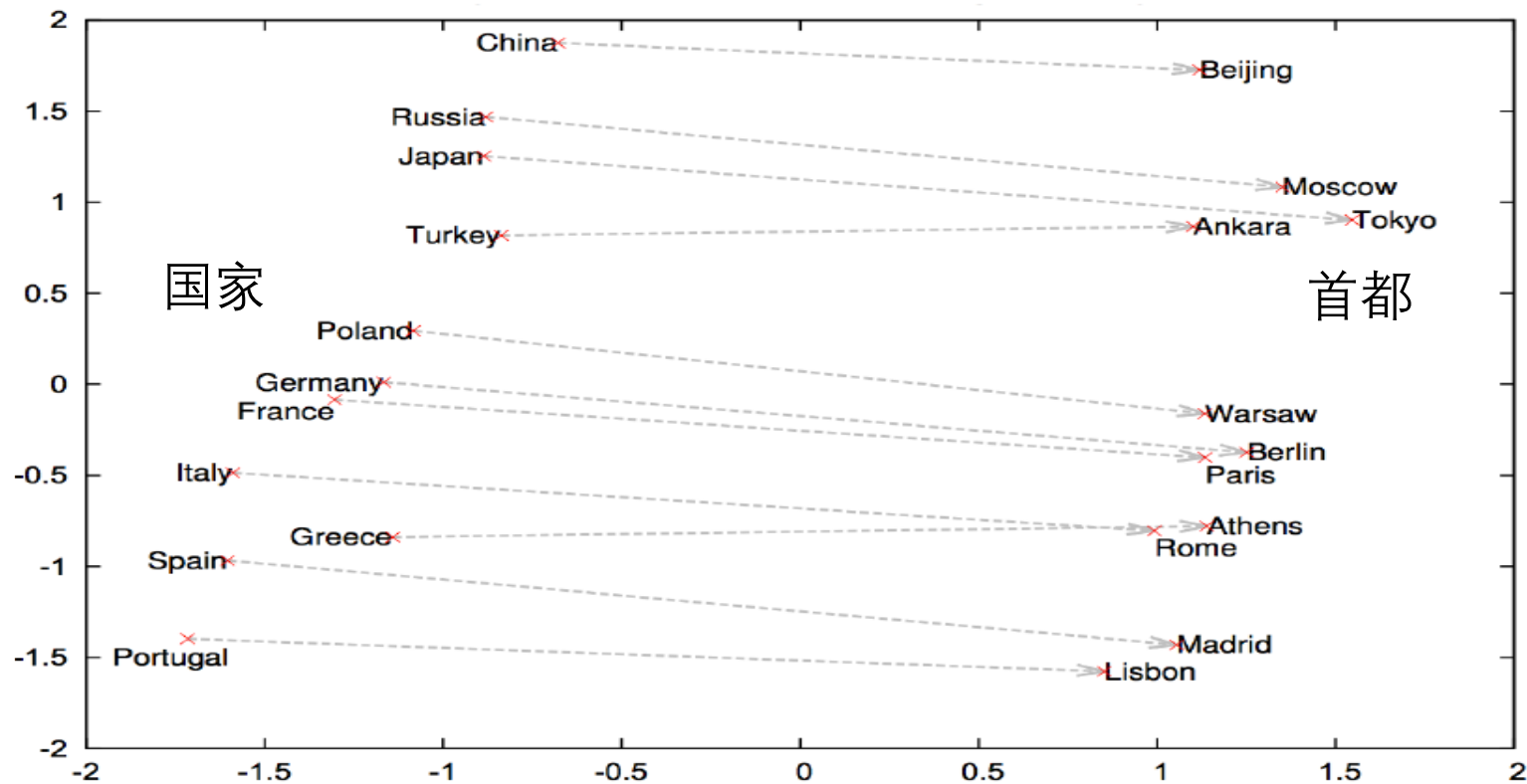
(语言) $\theta_{\text{france}} - \theta_{\text{french}} \approx \theta_{\text{mexico}} - \theta_{\text{spanish}}$

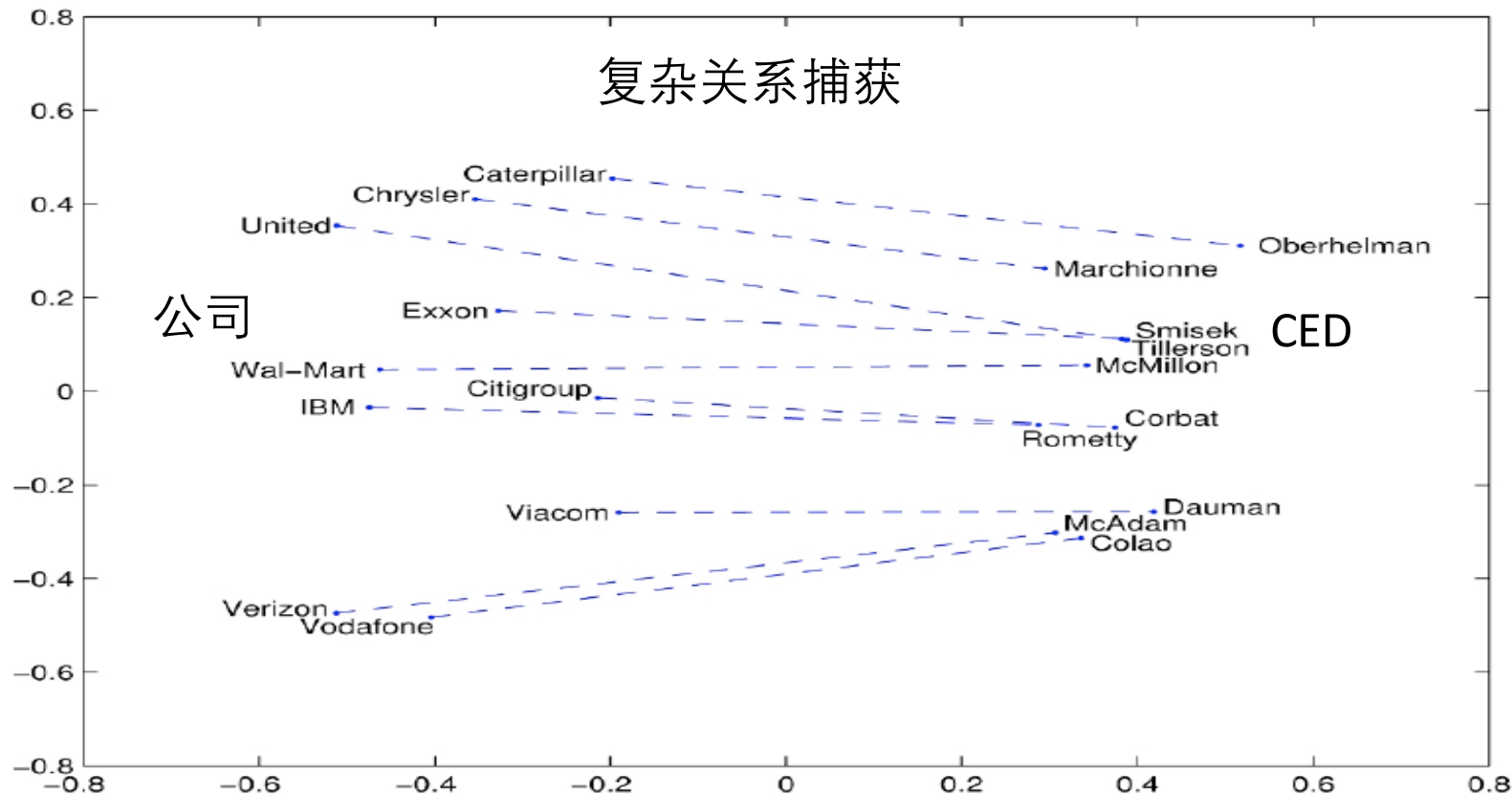
(复数形式) $\theta_{\text{car}} - \theta_{\text{cars}} \approx \theta_{\text{apple}} - \theta_{\text{apples}}$

直观解释

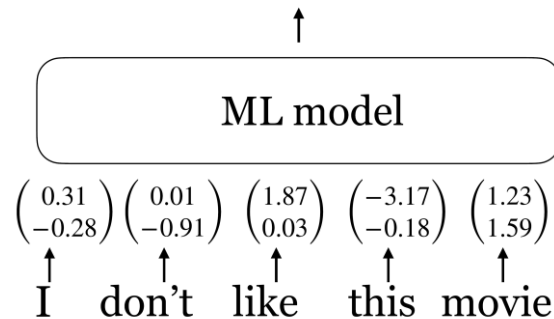
$$\underbrace{\theta_{\text{king}}}_{[\text{crown, he}]} - \underbrace{\theta_{\text{man}}}_{[\text{he}]} \approx \underbrace{\theta_{\text{queen}}}_{[\text{crown, she}]} - \underbrace{\theta_{\text{woman}}}_{[\text{she}]}$$

复杂关系捕获





- 外在评估 (extrinsic evaluation)
 - 把词向量作为NLP系统的输入, 看看是否提高了性能
 - 要经历过对原系统的重新训练和测试, 费时
 - 是最重要的评估手段
- 内在评估 (intrinsic evaluation)
 - 在一个特定的子任务上评估
 - 评估过程快
 - 但不能确定是否真的对下游NLP任务有帮助



Word similarity

Example dataset: wordsim-353

353 pairs of words with human judgement

<http://www.cs.technion.ac.il/~gabr/resources/data/wordsim353/>

Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92

Cosine similarity:

$$\cos(\mathbf{u}_i, \mathbf{u}_j) = \frac{\mathbf{u}_i \cdot \mathbf{u}_j}{\|\mathbf{u}_i\|_2 \times \|\mathbf{u}_j\|_2}.$$

Metric: Spearman rank correlation

Model	Size	WS353	MC	RG	SCWS	RW
SVD	6B	35.3	35.1	42.5	38.3	25.6
SVD-S	6B	56.5	71.5	71.0	53.6	34.7
SVD-L	6B	65.7	<u>72.7</u>	75.1	56.5	37.0
CBOW [†]	6B	57.2	65.6	68.2	57.0	32.5
SG [†]	6B	62.8	65.2	69.7	<u>58.1</u>	37.2
GloVe	6B	<u>65.8</u>	<u>72.7</u>	<u>77.8</u>	53.9	<u>38.1</u>
SVD-L	42B	74.0	76.4	74.1	58.3	39.9
GloVe	42B	<u>75.9</u>	<u>83.6</u>	<u>82.9</u>	<u>59.6</u>	<u>47.8</u>
CBOW [*]	100B	68.4	79.6	75.4	59.4	45.5

Word analogy test: $a : a^* :: b : b^*$

$$b^* = \arg \max_{w \in V} \cos(e(w), e(a^*) - e(a) + e(b))$$

semantic

syntactic

Chicago:Illinois $\approx$ Philadelphia: ? bad:worst $\approx$ cool: ?

More examples at

<http://download.tensorflow.org/data/questions-words.txt>

Metric: accuracy

Model	Dim.	Size	Sem.	Syn.	Tot.
ivLBL	100	1.5B	55.9	50.1	53.2
HPCA	100	1.6B	4.2	16.4	10.8
GloVe	100	1.6B	<u>67.5</u>	<u>54.3</u>	<u>60.3</u>
SG	300	1B	61	61	61
CBOW	300	1.6B	16.1	52.6	36.1
vLBL	300	1.5B	54.2	<u>64.8</u>	60.0
ivLBL	300	1.5B	65.2	63.0	64.0
GloVe	300	1.6B	<u>80.8</u>	61.5	<u>70.3</u>
SVD	300	6B	6.3	8.1	7.3
SVD-S	300	6B	36.7	46.6	42.1
SVD-L	300	6B	56.6	63.0	60.1
CBOW [†]	300	6B	63.6	<u>67.4</u>	65.7
SG [†]	300	6B	73.0	66.0	69.1
GloVe	300	6B	<u>77.4</u>	67.0	<u>71.7</u>
CBOW	1000	6B	57.3	68.9	63.7
SG	1000	6B	66.1	65.1	65.6
SVD-L	300	42B	38.4	58.2	49.2
GloVe	300	42B	<u>81.9</u>	<u>69.3</u>	<u>75.0</u>

- 神经语言模型
 - ✓ 背景
 - ✓ 前馈神经语言模型
 - ✓ 模型训练
- 词嵌入
 - ✓ 背景
 - ✓ 向量语义
 - ✓ Word2vec
- 讨论

- 前馈神经语言模型

- ✓突破：通过机器学习方式，学习出每个词的向量表示形式，这启发了后续的词嵌入（word embedding）的研究，是神经网络/深度学习技术进入自然语言处理的开创性工作
- ✓局限：仍然局限于马尔可夫假设，只能处理固定长度的输入，不能捕获超出上下文窗口的依存关系。为此，引发了研究循环神经网络等更为灵活的架构来构建语言模型

- 词嵌入

- ✓ 是一种静态的词语义向量表示方法，不能有效地解决一词多意的问题
- ✓不能真正体现出“词的意思由其上下文决定”
- ✓上下文窗口固定，无法捕获窗口之外的词与目标词之间的关系

循环网络语言模型